day09_网络进阶

今日内容：

- 【记录】1- RAID的实战
- 【作业讲解】2- 作业处理
- 【理解+记录】3- OSI七层网络模型
- 【理解+记录】4- TCP/IP协议模型
- 【了解】5- VMware三种网络模式
- 【理解+实操】6- 路由表
- 【作业】7- 实战： 路由选择实战

1. OSI七层模型

1.1 基本介绍

OSI（Open Systems Interconnection，开放系统互联）七层模型是由国际标准化组织（ISO）提出的一个网络通信参考模型，旨在为网络协议的设计和互操作提供指导。OSI 模型将网络通信划分为七个层级，每一层都有特定的功能，分工明确并相互协作。

> OSI 七层模型是用来帮助我们理解网络是怎么传输数据的。它是国际标准化组织（ISO）设计的一套规则，就像盖房子的蓝图一样，把复杂的网络通信分成七个部分，每个部分有自己负责的任务。这样，网络工程师和设备厂商就能按照这个标准来设计和改进网络，保证不同的设备、软件都能互相兼容、正常工作。简单来说，这个模型就像一部说明书，指导我们怎么让网络中的各种东西配合得更好。

开放系统，实质上指的是遵循OSI参考模型和相关协议能够实现互连的具有各种应用目的的计算机系统。

那么七层模型有哪些呢？

物理层、数据链路层、网络层、传输层、会话层、表示层、应用层



1.2 模型细化介绍




```
1  1- 物理层：
2        功能：在网络设备之间传输原始比特流(0 和 1)，包括定义物理设备的硬件规格（如电缆类型、信号电压、传输速率等）
3        设备：网线、光纤、集线器
4        大白话：就是网线、光纤、无线信号这些东西，负责把 0 和 1 通过电流、光信号或者无线电波传输出去
5
6  2- 数据链路层：
```

功能：将比特组合成帧，并提供可靠的数据传输(访问控制和硬件地址(MAC 地址)管理)，检测和纠正物理层的错误

设备：交换机

大白话：把 0 和 1 组合成“数据包”，并且加上地址标签（MAC 地址），还会检查有没有传错。

同一局域网内传输操作，无法进行跨网络传输

3- 网络层：

功能：负责路径选择（路由）和逻辑地址（IP 地址）的管理，实现数据包的传输,主要的协议为 TPv4、IPv6

设备：路由器

大白话：负责导航，给数据找一条从A点到B点的最佳路径，用的是IP地址

基于路由器，实现跨网络的传输方式

4- 传输层：

功能：提供端到端的通信服务，数据从应用层分割成报文段，确保数据的可靠性、顺序性和完整性,主要的协议为 TCP、UDP

大白话：确保数据不丢、不乱，像快递的分拣中心，还会给大数据分成小包裹（分段），送达后再重新组装。

5- 会话层：

功能：管理会话（Session），包括建立、维护和终止应用程序之间的连接

例如：比如视频会议时确保两边能持续连通

大白话：就是维持聊天记录的管家，负责建立、管理和结束两个设备之间的连接。

6- 表示层：

功能：处理数据的表示形式，确保数据在不同系统之间的兼容性 负责数据的编码、加密、压缩等

例如：图片、视频、文字能正确显示都依靠表示层

大白话：数据翻译官，负责把数据变成对方能读懂的格式，还能帮忙加密或压缩。

7- 应用层：

功能：为用户和应用程序提供网络服务接口

例如：网页浏览（HTTP）、邮件（SMTP）、文件传输（FTP）

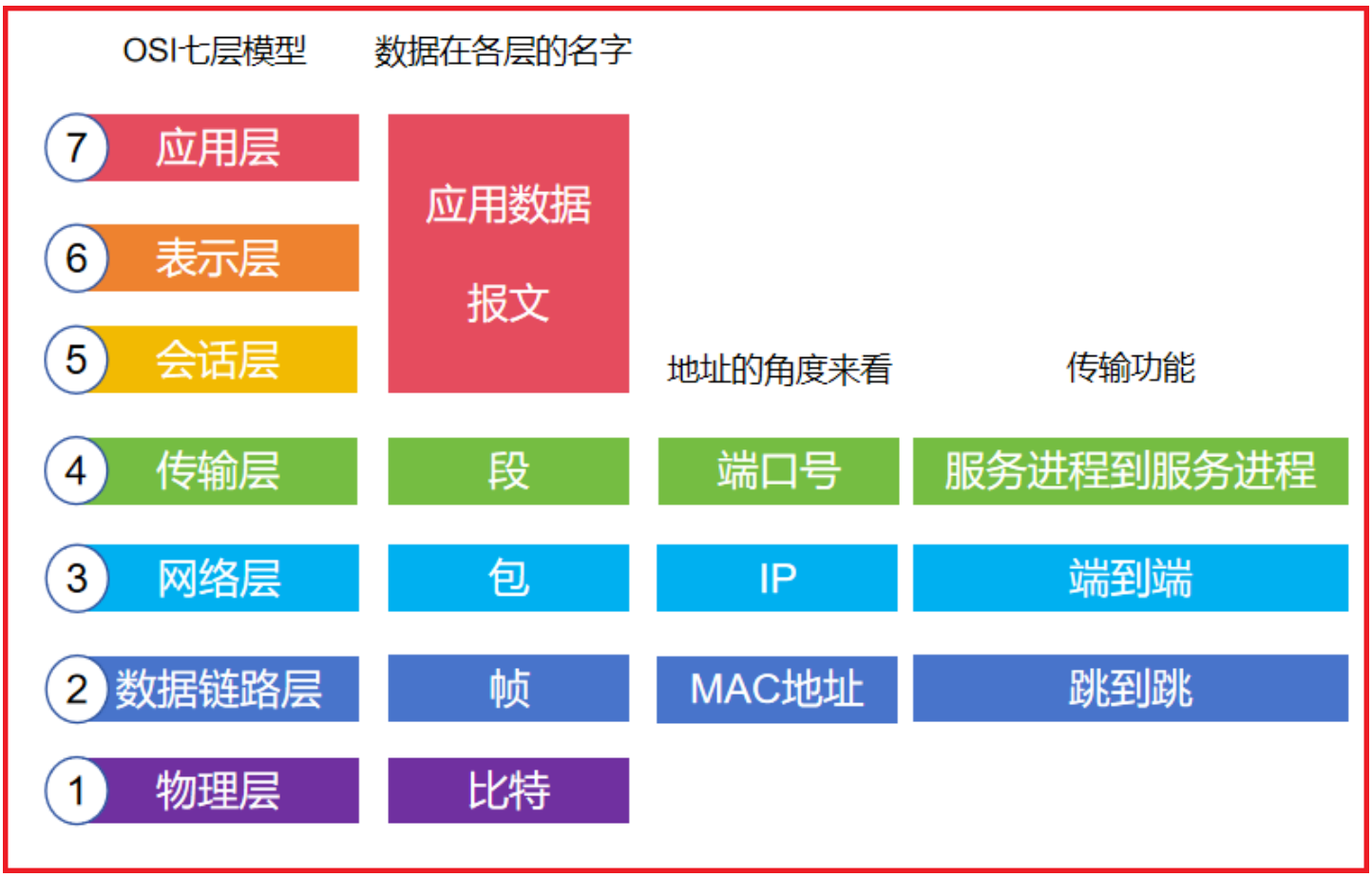
大白话：就是和你直接打交道的界面，负责提供网络服务

数据在各层的名字：

比特流 --> 帧 --> 包 -- 报文段 --> 报文（应用层、表示层、会话层）

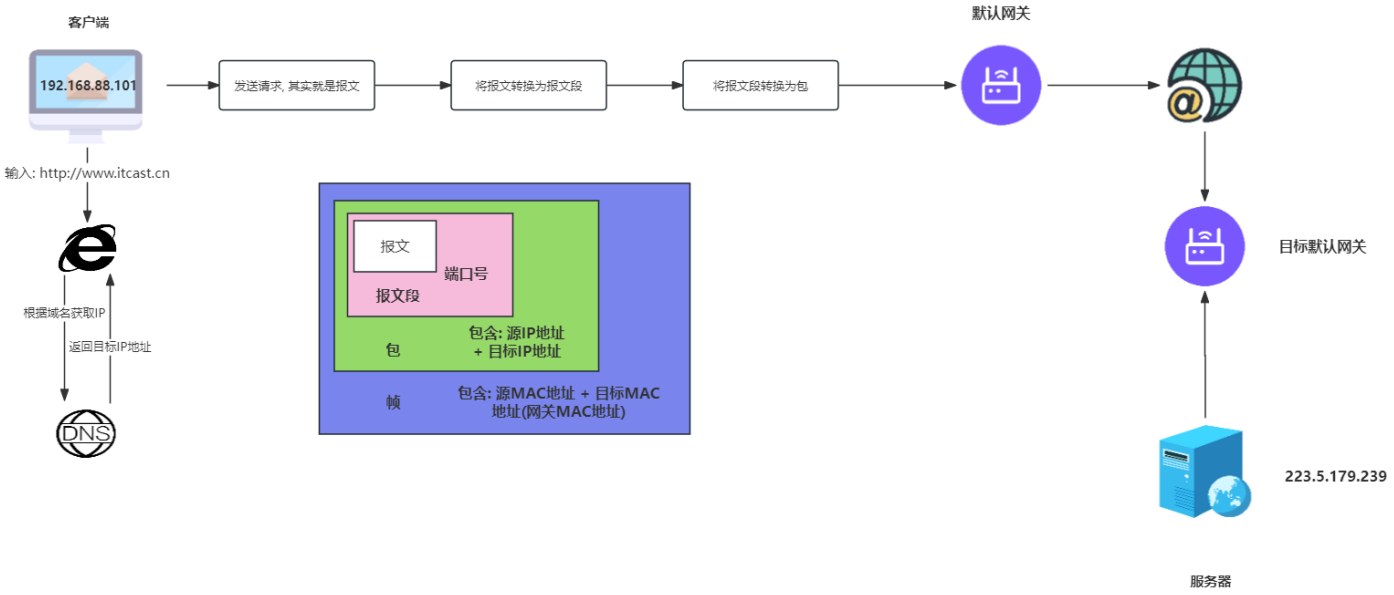
从地址角度来看：

```
48          传输层：端到端传输     端口
49          网络层：  IP
50          数据链路层：  MAC
51
52  从传输的功能：
53          传输层：  从服务进程  到  服务进程
54          网络层：  从IP到IP   端到端
55          数据链路层：  从mac  到mac  跳
```




```
1       针对，我们对OSI七层模型的介绍，  数据在七层模型中逐步进行处理，  针对数据在各层的名字，在应用层、表示层和会话层的数据称为应用数据或者应用负载也可以叫上层数据，同时也是我们常听到的报文，  而在其他各层的名字分别为段、包、帧和比特。
2       从地址角度看，主要由端口号、IP和MAC地址
3       从传输功能角度看，  主要是服务进程到服务进程，端到端以及跳到跳
```

接下里，我们将所学的内容关联起来， 串在一起，客户端是如何一步一步从客户端访问到服务器的：




```
例如，当客户端通过浏览器访问http://www.itcast.cn， 从而连接传智教育官网服务器

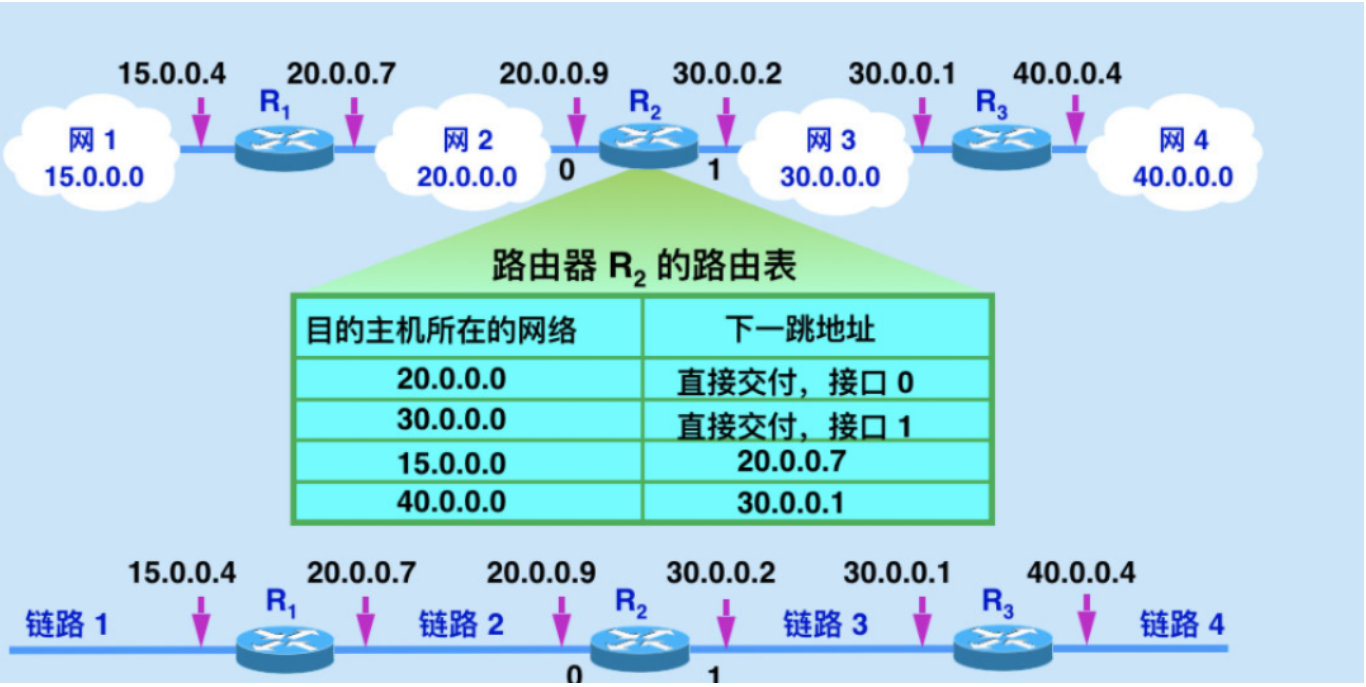
1- 客户端要发送数据，也就是报文:
        1.1 浏览器需要先通过DNS获取 www.itcast.cn 的 IP 地址
        1.2 浏览器生成一个 HTTP 请求报文,其包括有请求方式、请求路径地址、请求协议、请求主机地址（IP与域名和端口）、请求头信息，例如浏览器信息等
        1.3 如果是 HTTPS，则会在对 HTTP 请求报文进行 加密（SSL/TLS）,如果是 HTTP，则直接传递

2- 当报文来到传输层加上端口号，最终封装为数据段
        2.1 将报文分割成多个报文段（Segment）
        2.2 在每个报文段头部添加 传输层头部:
                源端口（如 12345，客户端随机生成）
                目标端口（如 80，用于 HTTP）
                序列号：标记数据顺序，确保接收方能按顺序组装
                校验号：用于检测传输中是否出错
        2.3 选择协议:
                TCP（可靠传输）：确保数据完整、无序、无丢失
                UDP（不可靠传输）：用于 DNS 查询

3- 当数据段来到网络层,加上IP地址，封装成包
        3.1 为报文段添加 网络层头部，生成 数据包（Packet）
                源 IP 地址：如 192.168.1.100（你的电脑）。
                目标 IP 地址：如 202.102.110.219（目标服务器）。
        3.2 路由器根据目标 IP 地址，规划数据包的传输路径
        3.3 如果目标在局域网外，数据包会被发往默认网关。

4- 当数据包来到数据链路层， 加上目标服务器MAC地址，封装成帧
        4.1 为数据包添加 数据链路层头部和尾部，生成 帧（Frame）
                源 MAC 地址：如 00:1A:2B:3C:4D:5E（你的网卡）。
```

29 目标 MAC 地址：网关的 MAC 地址（下一跳设备）
30 4.2 帧中还包括 帧校验序列（FCS），用于检测链路层的错误。
31
32
33 注意：我们发现包是包含目标IP地址的,因为我们要知道数据要发送到什么地址,但因为目标IP地址不是同一个网络下的,要发送到其他的网络就需要经过默认网关(也就是路由器地址)。 现在就出现了一个问题客户端主机最初并不知道默认网关的MAC地址没有办法封装成帧



1 如何找到目标MAC地址呢？ 中间就需要通过多层的路由器，完成逐层跳转，从而最终找到目标网络的MAC地址
2
3 在这个过程中， 就有一个比较重要的协议， ARP协议：是一个在局域网中用于解析 IP 地址到 MAC 地址 的协议。它的主要目的是通过目标设备的 IP 地址，找到对应的物理地址（MAC 地址），从而实现数据帧的正确传输
4
5 ARP工作原理：
6 主机 A 会在局域网中发送一个广播帧（ARP 请求），内容是：
7 内容一：谁是 192.168.1.1（默认网关地址）？请告诉我。
8 内容二：源 MAC 地址：主机 A 的 MAC 地址 00:11:22:33:44:55。
9 内容三：目标 MAC 地址：FF:FF:FF:FF:FF:FF（广播地址）。
10 内容四：数据部分：需要解析的 IP 地址（192.168.1.1）。
11
12 ARP 响应：
13 局域网内的设备都会收到这个广播帧，但只有默认网关（IP 地址为 192.168.1.1）会响应：
14 内容一： 我是 192.168.1.1，我的 MAC 地址是 AA:BB:CC:DD:EE:FF。
15 响应帧的源 MAC 地址是默认网关的 MAC 地址 AA:BB:CC:DD:EE:FF

```
16                          响应帧的目标 MAC 地址是主机 A 的 MAC 地址
    00:11:22:33:44:55
17          缓存结果：
18                  主机 A 将 192.168.1.1（默认网关地址） 的 MAC 地址
    AA:BB:CC:DD:EE:FF 缓存到 ARP 表（内存中）中
19                  之后的所有数据帧就可以直接使用这个 MAC 地址，而无需再次发送 ARP 请求
20
21
22  最后： 数据到达了目标网络路由， 如果目标的网关知道目标IP地址和MAC地址是哪台主机封装成帧
    就可以直接发送过去了，如果不知道也还是可以用ARP喊下街就好，目标主机收到包确认是自己的IP
    地址以后，解封查看段可以发现源和目标端口号，用目标端口号给到指定的应用程序，应用程序处理
    好以后，就可以按照源的信息做出响应，回去的原理也是一样的
```

2. TCP/IP协议模型

2.1 什么是TCP/IP协议

TCP/IP 协议是互联网中最重要的通信协议套件，定义了计算机之间如何在网络上进行通信。它包括一组相互关联的协议，分别处理数据的分段、传输、路由和应用层操作。

通俗解释

> TCP/IP 协议套件就像一套规则手册，指导设备如何相互通信、传递消息。

- 想象你寄快递：
  - **TCP** 确保包裹被正确拆分、传输，并按照顺序送达。
  - **IP** 负责规划路线，把包裹从发件地址送到收件地址。

TCP/IP 的全称：

- TCP：Transmission Control Protocol（传输控制协议）。
  - 确保数据可靠传输、按顺序到达、不丢失。
- IP：Internet Protocol（互联网协议）。
  - 负责数据的路由和寻址，确定数据的发送地址和目标地址。

TCP/IP 是一个协议套件：

- 除了 TCP 和 IP，还包括很多其他协议，比如：
  - **HTTP**：用于浏览网页。
  - **DNS**：用于域名解析。
  - **SMTP**：用于发送邮件。
  - **UDP**：用于快速传输（不保证可靠性）。

2.2 TCP/IP层级模型



2.3 TCP/IP三次握手与四次挥手

TCP全双工的协议（两端都可以发送数据也都可以接收数据， 两端可以同时发 也可以同时接）

了解相关名词:

```
序列号：seq序号，占32位，用来标识从TCP源端向目的端发送的字节流，发起方发送数据时对此进行标记。
确认序号：ack序号，占32位，只有ACK标志位为1时，确认序号字段才有效，Ack=Seq+1。

常见的标志位：
        ACK：确认序号有效。
        SYN：发起一个新连接。
        FIN：释放一个连接。
```

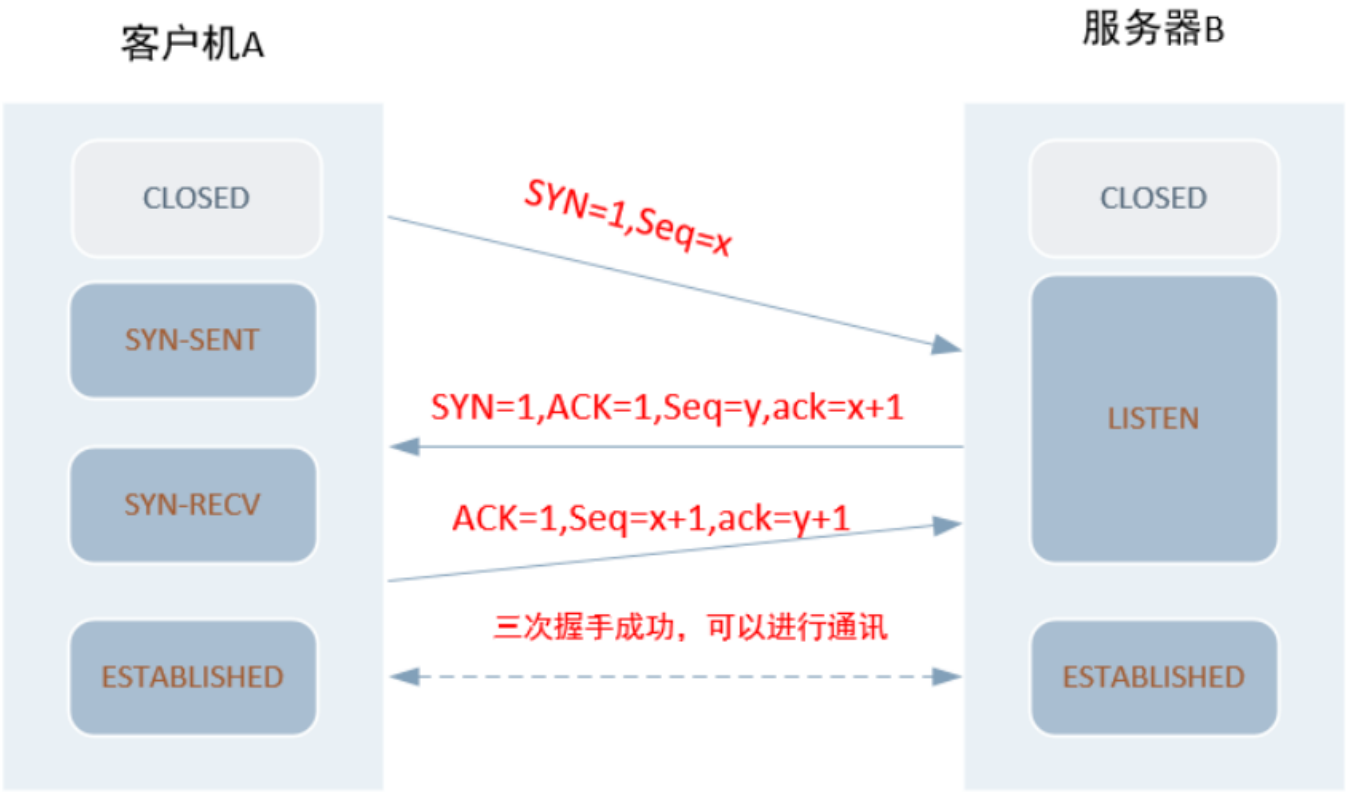
了解netstat中的网络状态

```
    CLOSED                          初始（无连接）状态。
    LISTEN                          侦听状态，等待远程机器的连接请求。
    SYN_SEND
    在TCP三次握手中，主动连接端发送了SYN包后，进入SYN_SEND状态，等待对方的ACK包。
    SYN_RECV
    在TCP三次握手中，主动连接端收到ACK包后，进入SYN_RECV状态。
    ESTABLISHED
    完成TCP三次握手后，主动连接端进入ESTABLISHED状态。此时，TCP连接已经建立，可以进行通信。

    FIN_WAIT_1      在TCP四次断开时，主动关闭端发送FIN包后，进入FIN_WAIT_1状态
    FIN_WAIT_2      在TCP四次断开时，主动关闭端收到ACK包后，进入FIN_WAIT_2状态
```

```
12      TIME_WAIT       在TCP四次断开时，主动关闭端发送了ACK包之后，进入TIME_WAIT状态
13      CLOSE_WAIT      在TCP四次断开时，被动关闭端收到FIN包后，进入CLOSE_WAIT状态
14      LAST_ACK        在TCP四次断开时，被动关闭端发送FIN包后，进入LAST_ACK状态，等待对
    方的ACK包。
```

2.3.1 三次握手

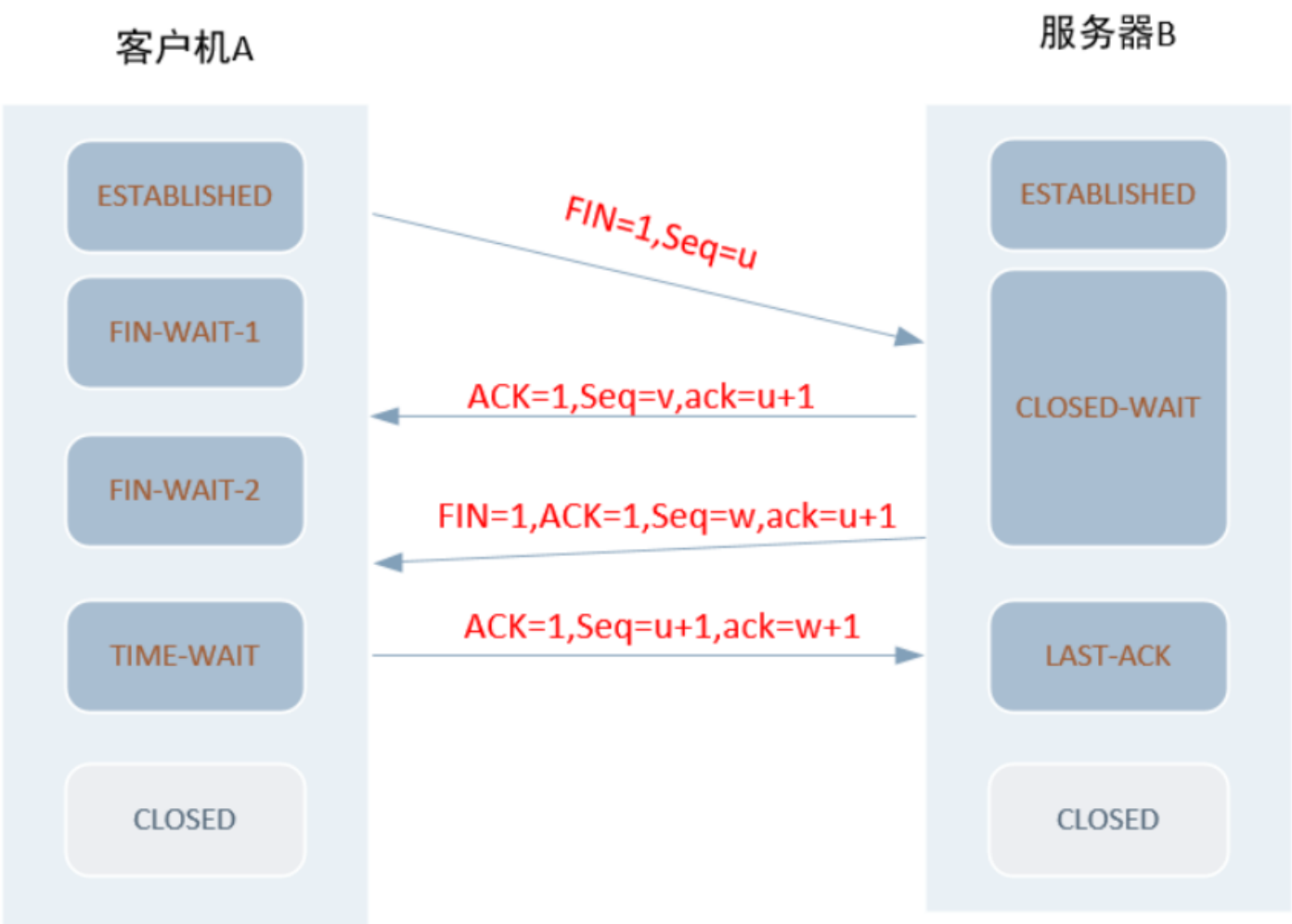


- **SYN（Synchronize）**：同步序列号，用于发起连接。
- **ACK（Acknowledgment）**：确认标志，表示确认号字段有效。
- **SEQ（Sequence Number）**：数据包的序列号，确保数据有序传输。
- **ACK Number**：期望收到的下一个字节的序列号（即已收到的序列号+1）。

```
1   TCP三次握手的过程如下：
2    1．客户机A端（主动连接端）发送一个SYN包给服务器B端（被动连接端）；
3    2．服务器B端（被动连接端）收到SYN包后，发送一个带ACK和SYN标志的包给客户机A端（主动连接
    端）；
4    3．客户机A端（主动连接端）发送一个带ACK标志的包给服务器B端（被动连接端），握手动作完
    成。
5
6
7   三次握手
8   1．客户端：我想发送请求
9   2．服务端：好的，我知道你要发送请求了，同意发送
10  3．客户端：好的，我知道你同意了，那我开始发送了
```

```
11
12
13  三次握手：
14  第一次握手： 客户端发送请求到服务器端， 要求建立TCP连接
15  第二次握手： 服务器端接收到客户建立TCP连接的请求， 并做出响应告知已经收到连接请求， 确认客户端是否可以收到服务器的响应信息
16  第三次握手： 客户端收到服务器的响应信息后，做出响应，告知服务器端已经收到响应信息
17
18  这个时候， 不管是客户端还是服务端，都确认对应可以发送消息也可以接受消息， 此时建立好TCP连接通道
19
20  后续就可以正常发送消息，但是要注意， 后续每一次发送消息其实都在存在再次确认方案
21
22
23
24  TCP是全双工模式：两端都可以发 也都可以同时接收
```

2.3.2 四次挥手




```
1  TCP四次断开的过程如下：
2  1. 客户机A端（主动连接端）发送一个FIN包给服务器B端（被动连接端）请求断开连接；
3  2. 服务器B端（被动连接端）收到FIN包后，发送一个ACK包给客户机A端（主动连接端）；
```

3．服务器B端（被动连接端）发送了ACK包后，再发送一个FIN包给客户机A端（主动连接端）确认断开；
4．客户机A端（主动连接端）收到FIN包后，发送一个ACK包，当服务器B端（被动连接端）收到ACK包后，四次断开动作完成，连接断开。

四次挥手
1．客户端：我不需要请求了，可以关闭连接了
2．服务端：好的，我知道你要关闭了
3．服务端：我发送完成了，可以关闭了
4．客户端：好的，我知道你发送完成了，确认关闭

四次挥手：
第一次挥手： 客户端发起断开请求（客户端已经发送完数据了， 想要断开连接）
第二次挥手： 服务端收到要断开请求的连接， 响应客户端， 告知已经收到断开请求， 但需要等待我这边发送完成
第三次挥手： 服务端在完成数据发生后， 再次响应给客户端， 告知已经发送完成， 可以进行关闭操作
第四次挥手： 客户端一直在等待接收服务器端告知已经发送完成的请求， 一旦收到后， 就理解响应服务器告知已经关闭完成，服务端也会正常关闭

通过这四次挥手，可以确保客户端 和 服务器都能够保证双方都发送完了消息也接收完了消息， 从而正常断开连接

要求一： 能够说出OSI和 TCP/IP的 七层模型和四层模型

要求二： 能够说出OSI的七层模型每一层的作用

要求三：能够说出网络通信客户端是如何一步一步连接到服务器端

要求四：TCP三次握手 和 四次挥手能讲的出来

希望可以用自己的话来进行阐述， 千万不要感觉出来你在背诵一个内容

先按照这些问题， 一一整理好每个问题的答案， 然后后续使用多天 多次的查看即可

3. VMware网络模式

3.1 虚拟设备

VMnet0：用于虚拟**桥接网络**下的虚拟交换机

VMnet1：用于虚拟**Host-Only网络**下的虚拟交换机

VMnet8：用于虚拟**NAT网络**下的虚拟交换机

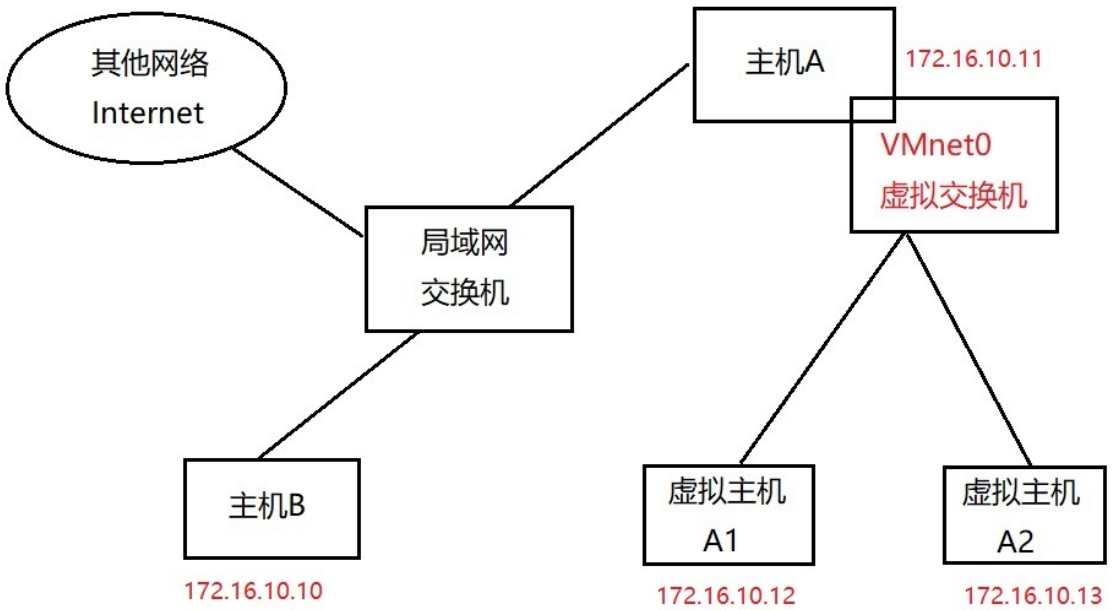
VMware Network Adepter VMnet1：Host用于与Host-Only虚拟网络进行通信的虚拟网卡

VMware Network Adepter VMnet8：Host用于与NAT虚拟网络进行通信的虚拟网卡

3.2. 三种网络模式

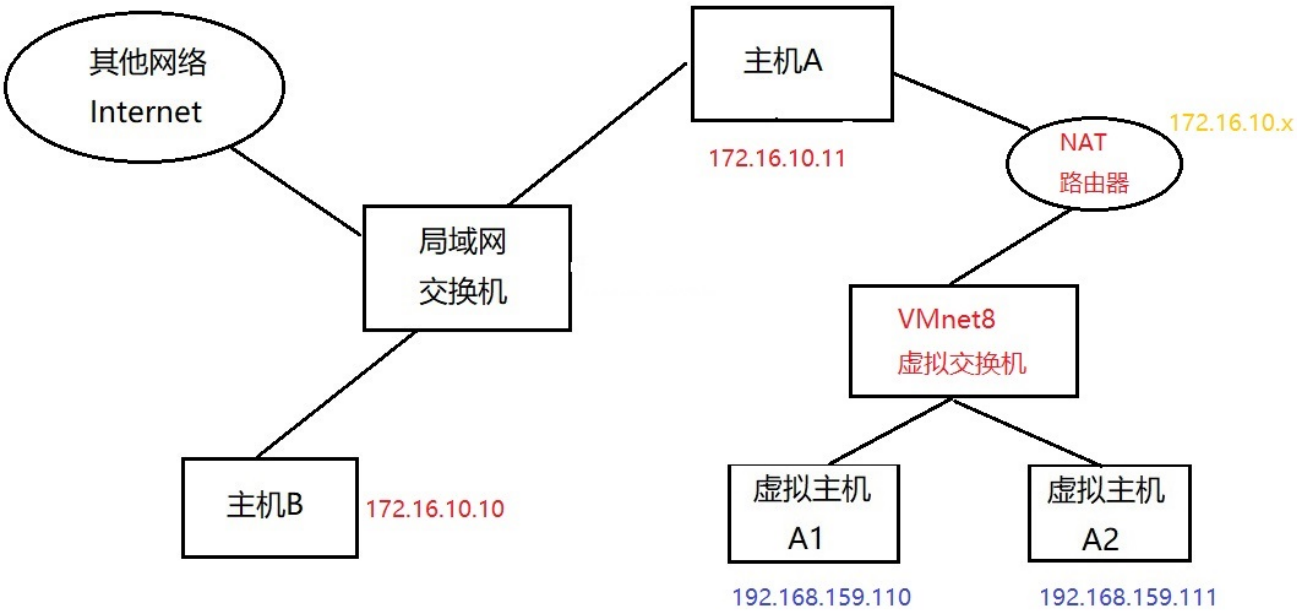
- 桥接网络

> 桥接网络是指虚拟网卡通过VMnet0虚拟交换机和本地物理网卡进行桥接，那么物理网卡和虚拟网卡就相当于处于同一个网段，虚拟交换机就相当于一台现实网络中的交换机。所以要想虚拟机也可以连接到互联网中，那么两个网卡的IP地址也要设置为同一网段。



- NAT网络

> 在NAT网络中，会用到VMware Network Adepter VMnet8虚拟网卡，主机上的VMware Network Adepter VMnet8虚拟网卡被直接连接到VMnet8虚拟交换机上与虚拟网卡进行通信。VMware Network Adepter VMnet8虚拟网卡的作用仅限于和VMnet8网段进行通信，它不给VMnet8网段提供路由功能，所以虚拟机虚拟一个NAT服务器，使虚拟网卡可以连 接到Internet。 VMware Network Adepter VMnet8虚拟网卡的IP地址是在安装VMware时由系统指定生成的，我们尽量不要修改这个数值，否则可能会使主机和虚拟机无法通信。

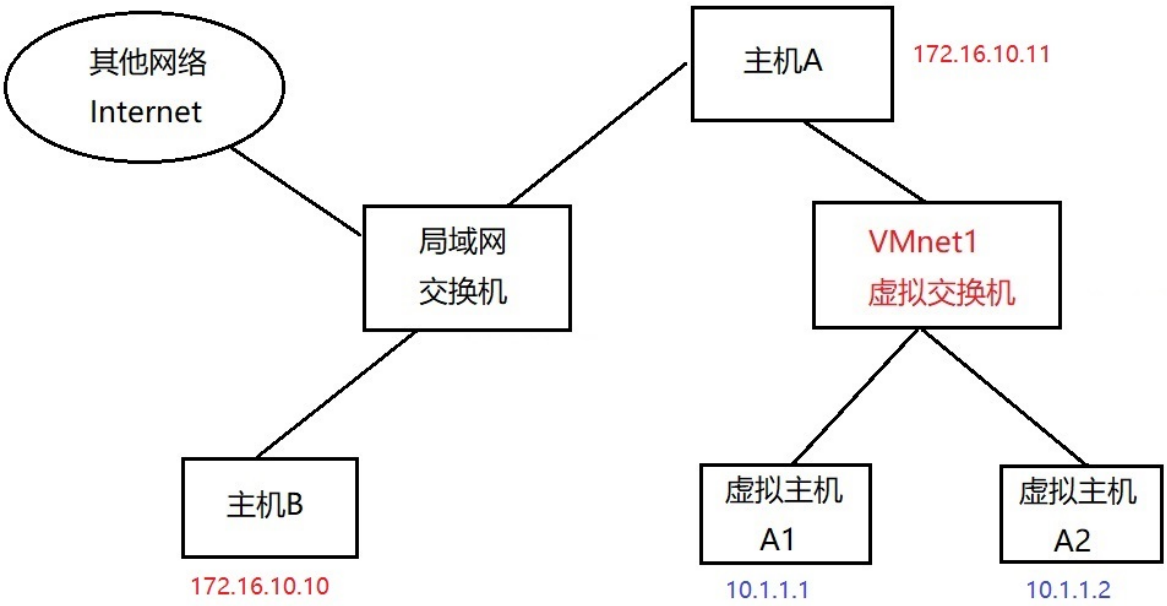


思考：

1. 虚拟主机A1是否可以访问物理真机主机B?
2. 虚拟主机A1是否可以访问物理真机主机 B 下的NAT模式下的虚拟机?

Host-Only模式

- 在Host-Only模式下，虚拟网络是一个全封闭的网络，它唯一能够访问的就是物理真机。其实Host-Only网络和NAT网络很相似，不同的地方就是Host-Only网络没有NAT服务，所以虚拟网络不能连接到Internet。主机和虚拟机之间的通信是通过VMware Network Adepter VMnet1虚拟网卡来实现的。



总结：

1. vmware产品会有哪些网络设备
   - 三个虚拟交换机
   - 两张虚拟网卡——>仅主机模式和NAT模式——>作用：用于物理真机和虚拟机之间连接
2. 三种网络模式
   - 桥接模式
     - 虚拟机的IP和物理真机在同一个网段
     - 默认是可以访问互联网的
   - NAT模式
     - 虚拟机的IP和物理真机不在同一个网段，一般私有地址
     - 默认也是可以访问互联网的，why？——>因为虚拟交换机上有一个nat路由器（地址转换功能）
   - 仅主机模式
     - 虚拟机的IP和物理真机不在同一个网段，一般私有地址
     - 默认是不可以访问互联网的

4. 路由表

什么是交换,什么是路由,什么是路由表?

```
1   1 交换是指 同网络访问（两台机器连在同一个交换机上，配置同网段的不同IP就可以直接通迅）
```

```
2
3   2 路由就是跨网络访问（路径选择）
4
5   3 路由表是记录路由信息的表，在Linux中首先是一张可见的，可更改的表,它的作用就是当数据包发到Linux系统的时候,系统（或者说内核)就根据这张表中定义好的信息来决定这个数据包接下来该怎么走
```

4.1 查看路由表信息

查看和设置路由表信息：

```
1   ip route show
2
3   这是现在Linux系统使用查看路由表的命令
```

```
[root@node1 ~]# ip route show
default via 192.168.88.2 dev ens160 proto static metric 100
192.168.88.0/24 dev ens160 proto kernel scope link src 192.168.88.101 metric 100
```

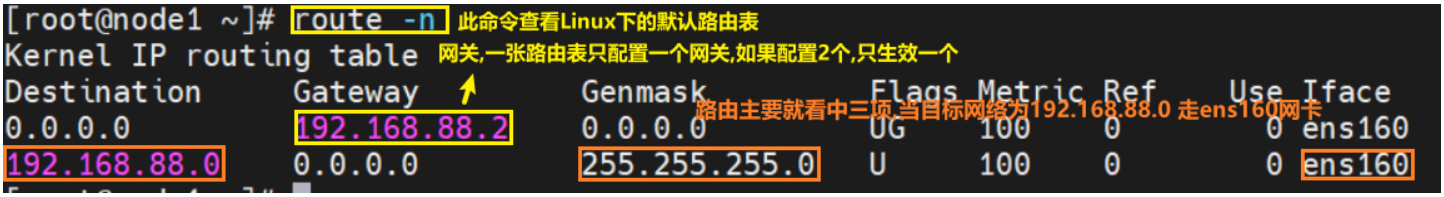
```
1   default via 192.168.88.2: 默认网关地址为 192.168.88.2  这是默认路由。当系统不知道数据应该发往哪里时，就会走这条路由。
2   metric 100 : 是路由的优先级。数值越小，优先级越高 如果有多条默认路由，系统会选择优先级最高（数值最小）的一条。
3
4   192.168.88.0/24 dev ens160: 网络 192.168.88.0/24 通过设备 ens160
5   proto kernel: 表示这条路由是系统内核自动添加的，通常是直接连接的网络
6   scope link: 表示这条路由适用于本地链路范围内的通信（同一子网内的设备）
7   src 192.168.1.100: 源 IP 地址
8
9   第一条信息:
10          任何找不到更具体目标的网络流量都会通过网卡 ens160 发送到 192.168.88.2，由它负责转发到其他网络。
11
12  第二条信息:
13          系统通过网卡 ens160 直接与 192.168.88.0/24 子网中的设备通信，无需网关转发。
```

---

route命令用来查看和设置路由表信息

```
1   # route -n                 //查看路由表信息
2
```

```
# 由于 route属于较旧的 net-tools 工具包，默认情况下不会预装
dnf -y install net-tools
```



输出项 说明：

Destination	目标网段或者主机
Gateway	网关地址，`*` 表示目标是本主机所属的网络，不需要路由
Genmask	网络掩码
Flags	标记。一些可能的标记如下：
	U — 路由是活动的
	H — 目标是一个主机
	G — 路由指向网关
	R — 恢复动态路由产生的表项
	D — 由路由的后台程序动态地安装
	M — 由路由的后台程序修改
	! — 拒绝路由
Metric	路由距离，到达指定网络所需的中转数（Linux 内核中没有使用）
Ref	路由项引用次数（Linux 内核中没有使用）
Use	此路由项被路由软件查找的次数
Iface	该路由表项对应的输出接口

4.2 读懂路由信息

```
[root@centos7 ~]# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.1.1.2        0.0.0.0         UG    0      0        0 ens33
10.1.1.0        0.0.0.0         255.255.255.0   U     0      0        0 ens33
169.254.0.0     0.0.0.0         255.255.0.0     U     1002   0        0 ens33
```

问题1: 按上面的路由表来看，如果要访问10.1.1.10这个IP，请问会怎么走?

查询路由表， 发现该IP对应的网段在目标路由存在的， 此时之间走 10.1.1.0 这条路由， 最终经由 ens33网卡 出去

问题2: 按上面的路由表来看，如果要访问14.200.151.38这个IP，请问会怎么走?

查询路由表，现该IP对应的网段或者主机在目标路由上并不存在，此时会走默认0.0.0.0所捕获，经由 ens33网卡 出去， 通过10.1.1.2 网关 连接到 外部14.200.151.38

当在一台Linux机器上要访问一个目标IP时，请记住Linux**四步口诀**:

① 如果本机有目标IP，则会直接访问本地；如果本地没有目标IP，则看第2步

② 用route -n查看路由，如果路由条目里包含了目标IP的网段，则数据包就会从对应路由条目后面的网卡出去

③ 如果没有对应网段的路由条目，则全部都走网关

④ 如果网关也没有，则报错：网络不可达

> 注意: 当不能直接到达目标IP， 那么每到达一个机器都会重复上面四步，直到找到目标)

问题3：如何加网关和删除网关，加网关有什么要求?

答:

```
临时网关:
# ip route add default via x.x.x.x dev ens33
# ip route del default via x.x.x.x dev ens33
或者
# route add default gw  x.x.x.x                    临时加网关，马上生效
# route del default gw  x.x.x.x                 临时删网关，马上生效



永久增加网关:
新版本:
# vim /etc/NetworkManager/system-connections/ens160.nmconnection
gateway=x.x.x.x

老版本
# vim /etc/sysconfig/network-scripts/ifcfg-ens33
GATEWAY=x.x.x.x

注意事项:
1. 加网关只能加你已经有的路由网段里的一个IP才行（此IP不一定存在）
2. 加网关可以不用指定子网掩码（因为是已有的一个网段的IP，所以掩码已经确认了）
```

4.3 路由类型

- 主机路由

  主机路由是路由选择表中指向单个IP地址或主机名的路由记录。主机路由的Flags字段为H。

  例如，在下面的示例中，本地主机通过IP地址192.168.1.1的路由器到达IP地址为10.0.0.10的主机。

```
Destination     Gateway         Genmask         Flags   Metric  Ref    Use Iface
-----------     -------         -------         -----   ------  ---    --- -----
10.0.0.10       192.168.1.1     255.255.255.255 UH      0       0        0 ens33
```

案例：添加到主机的路由

```
假设需要将目标主机 192.168.88.103 的流量通过网关 192.168.88.2，使用接口 ens160：

新版本：
ip route add 192.168.88.103 via 192.168.88.2 dev ens160

老版本：
# route add -host 192.168.88.103 dev ens160
# route add -host 192.168.88.103 gw 192.168.88.2      # 添加到192.168.88.2的网关
```

- 网络路由
  - 网络路由是代表主机可以到达的网络。网络路由的Flags字段为N。
  - 例如，在下面的示例中，本地主机将发送到网络192.19.12.0的数据包转发到IP地址为192.168.1.1的路由器。

```
Destination     Gateway         Genmask         Flags   Metric  Ref    Use Iface
-----------     -------         -------         -----   ------  ---    --- -----
192.19.12.0     192.168.1.1     255.255.255.0     UN      0       0        0 ens33
```

案例：添加到网络的路由

```
需求:添加192.168.16.0/24的网络
新版本：
ip route add 192.168.16.0/24 dev ens160

老版本：
```

```
添加192.168.16.0/24的网络
route add -net 192.168.16.0 netmask 255.255.255.0 ens160

需求:添加10.20.30.48/29的网络
新版本:
ip route add 10.20.30.48/29 dev ens160

老版本:
添加10.20.30.48的网络
route add -net 10.20.30.48/29 gw 192.168.88.2
```

- 默认路由

  当主机不能在路由表中查找到目标主机的IP地址或网络路由时，数据包就被发送到默认路由（默认网关）上。默认路由的Flags字段为G。例如，在下面的示例中，默认路由是IP地址为192.168.1.1的路由器。

```
Destination     Gateway         Genmask         Flags   Metric  Ref    Use Iface
-----------     -------         -------         -----   ------  ---    --- -----
default         192.168.1.1     0.0.0.0         UG      0       0        0 ens33
```

案例：添加默认路由

```
新版本:
ip route add default via 192.168.88.2 dev ens160

老版本
route add default gw 192.168.88.2
```

如何删除:

```
新版本:
ip route del ip/N

老版本:
# route del -host 192.168.88.103

# route del -net 10.20.30.40 netmask 255.255.255.248
# route del -net 10.20.30.48/29

```

```
# route del default
```

总结内容 【重点关注此部分】

```
主机路由指令:
    新版本:
        ip route add ip地址 [ via 网关 | dev 网卡设备]
        ip route del ip地址

    老版本:
        route add -host ip地址 [ gw 网关 | dev 网卡设备 ]
        route del -host ip地址

    注意:
        主机路由是针对某一个具体的目标IP地址设置路由转发， 转发的时候， 如果要转发到外部的网络， 请使用网关方案， 如果是转发到内部的网络 请使用 网卡设备方案


网络路由指令
    新版本:
        ip route add 网络地址/掩码位数 [ via 网关 | dev 网卡设备]
        ip route del 网络地址/掩码位数

    老版本:
        route add -net 网络地址/掩码位数 [ gw 网关 | dev 网卡设备 ]
        route del -host 网络地址/掩码位数

     注意:
        网络路由是针对某一个网段上N多台网络设备设置路由转发， 转发的时候， 如果要转发到外部的网络， 请使用网关方案， 如果是转发到内部的网络 请使用 网卡设备方案


默认网关指令:
     新版本:
        ip route add default via 网关地址
        ip route del default via 网关地址

    老版本:
        route add default gw 网关地址
        route del default gw 网关地址

    注意:
        如果在路由表中没有找打匹配的条目 统一匹配到默认网关， 有默认网关来完成数据转发工作
```

4.4 设置包转发

在 CentOS Stream 9 中默认的内核配置已经包含了路由功能，但默认并没有在系统启动时启用此功能。开启 Linux 的路由功能可以通过调整内核的网络参数来实现。要配置和调整内核参数可以使用 sysctl 命令。例如：要开启 Linux 内核的数据包转发功能可以使用如下的命令。

```
sysctl -w net.ipv4.ip_forward=1
```

这样设置之后，当前系统就能实现包转发，但下次启动计算机时将失效。为了使在下次启动计算机时仍然有效，需要将下面的行写入配置文件/etc/sysctl.conf。

```
vim /etc/sysctl.conf

# 添加以下内容:
net.ipv4.ip_forward = 1
```

用户还可以使用如下的命令查看当前系统是否支持包转发。

```
sysctl net.ipv4.ip_forward

0 表示不支持
1 表示支持
```

```
[root@node1 ~]# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

5. 路由选择实验

5.1 实验要求

```
前提条件：三台虚拟主机的网络模式都是仅主机模式!
环境准备:
jumper:192.168.1.1/24
    网络地址：192.168.1.0
    子网掩码：255.255.255.0
```

```
作为网关服务器，开启路由转发功能/proc/sys/net/ipv4/ip_forward

node1:10.1.1.1/24
    网络地址： 10.1.1.0
    子网掩码：255.255.255.0
node2:172.16.0.1/24
    网络地址：172.16.0.0
    子网掩码：255.255.255.0

要求：
    实现不同网络(10.1.1.1和172.16.0.1)之间的互通，使用第三方主机jumper作为路由进行转发
```

思路：

1. 中间人jumper，必须开启路由转发功能；
2. 中间人jumper即要有到达A网络，又能到达C网络的路
3. node1主机（A网络）必须设置中间人jumper作为自己的网关
4. node2主机（C网络）必须设置中间人jumper作为自己的网关

准备工作：

```
1- 由于三台服务器均为三个不同网络的中主机， 所以对应他们的交换机也不同网络的交换机，
    需要准备三个仅主机的交换机，并且三个仅主机的交换机网络地址分别为
        网络地址：192.168.1.0/24
        网络地址： 10.1.1.0/24
        网络地址：172.16.0.0/24

    思考：如何在VMware中增加三个交换机呢？ VMware --> 网络编辑器中看到的网络信息其实就是虚拟交换机

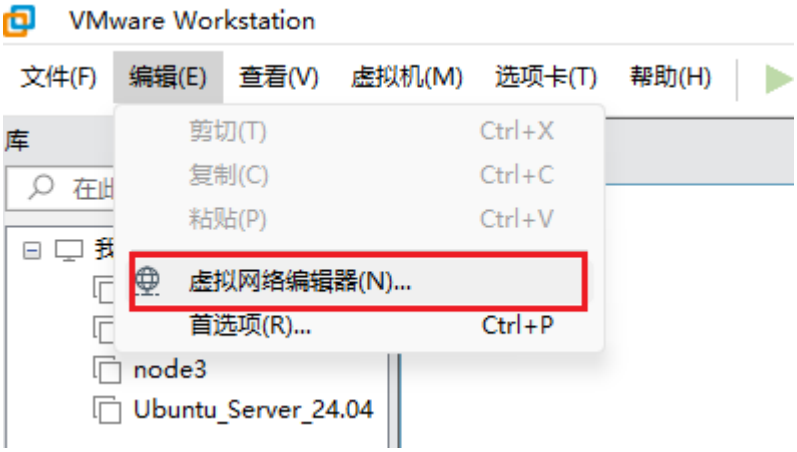
2- 准备三台服务器（建议选择克隆node1）， 要求这三台服务器的网络模式均为仅主机模式， 并且每台服务器对应的交换机应该是其对应网络的交换机
    node1的IP地址： 10.1.1.1/24   应该连接的交换机网卡为网络地址是10.1.1.0/24所对应的交换机网络
    node2的IP地址： 172.16.0.1/24 应该连接的交换机网卡为网络地址是172.16.0.0/24所对应的交换机网络
    jumper的IP地址：192.168.1.1/24 应该连接的交换机网卡为网络地址是192.168.1.0/24所对应的交换机网络

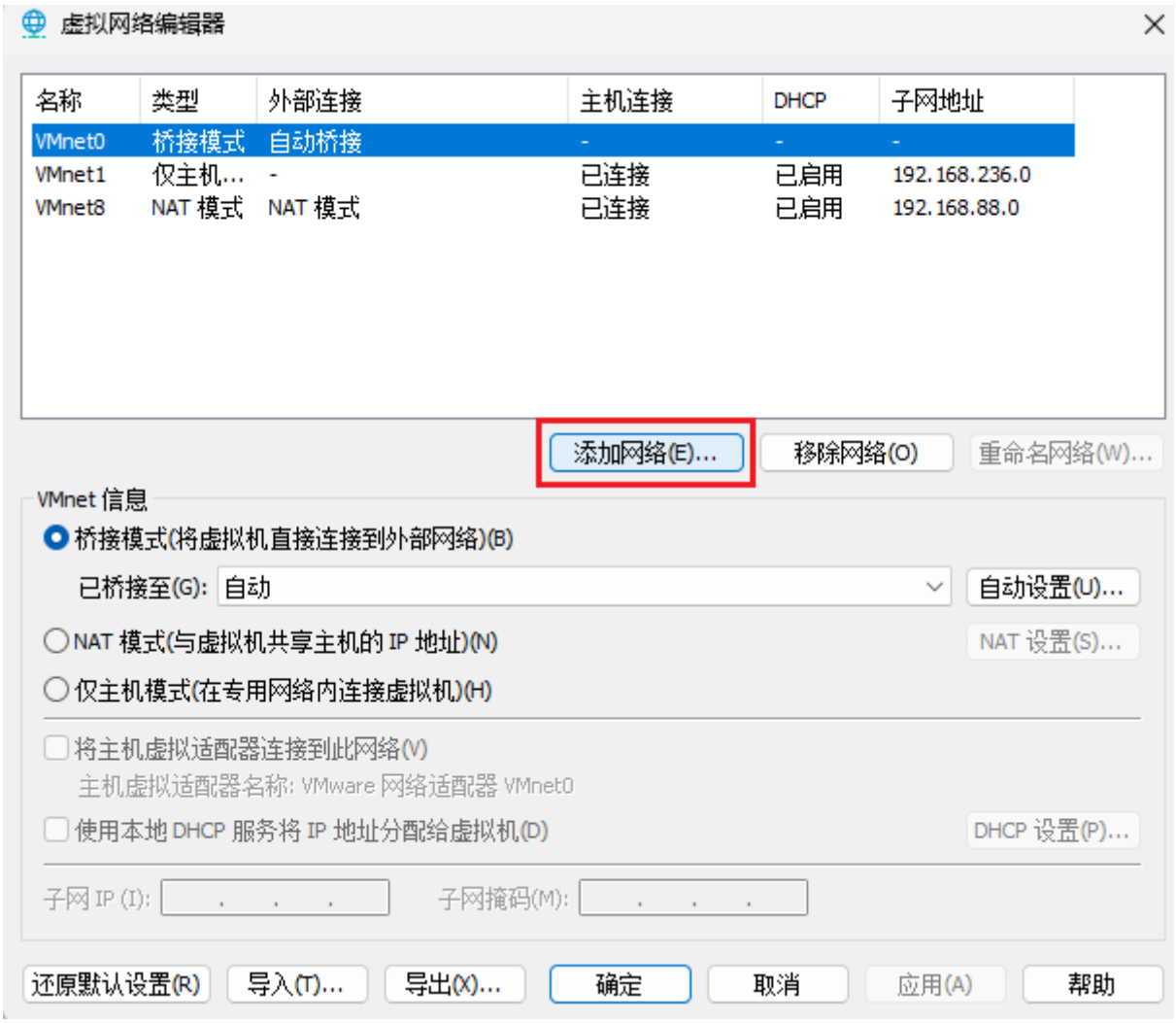
3- 开机后， 需要调整每个服务器的静态IP地址， 分别为 对应需求要求IP地址即可，在仅主机模式下， 不存在网关地址，所以当前无需配置网关（或随意给个值）

```

```
16    4- 配置完成后， 尝试使用MX工具连接成功
```

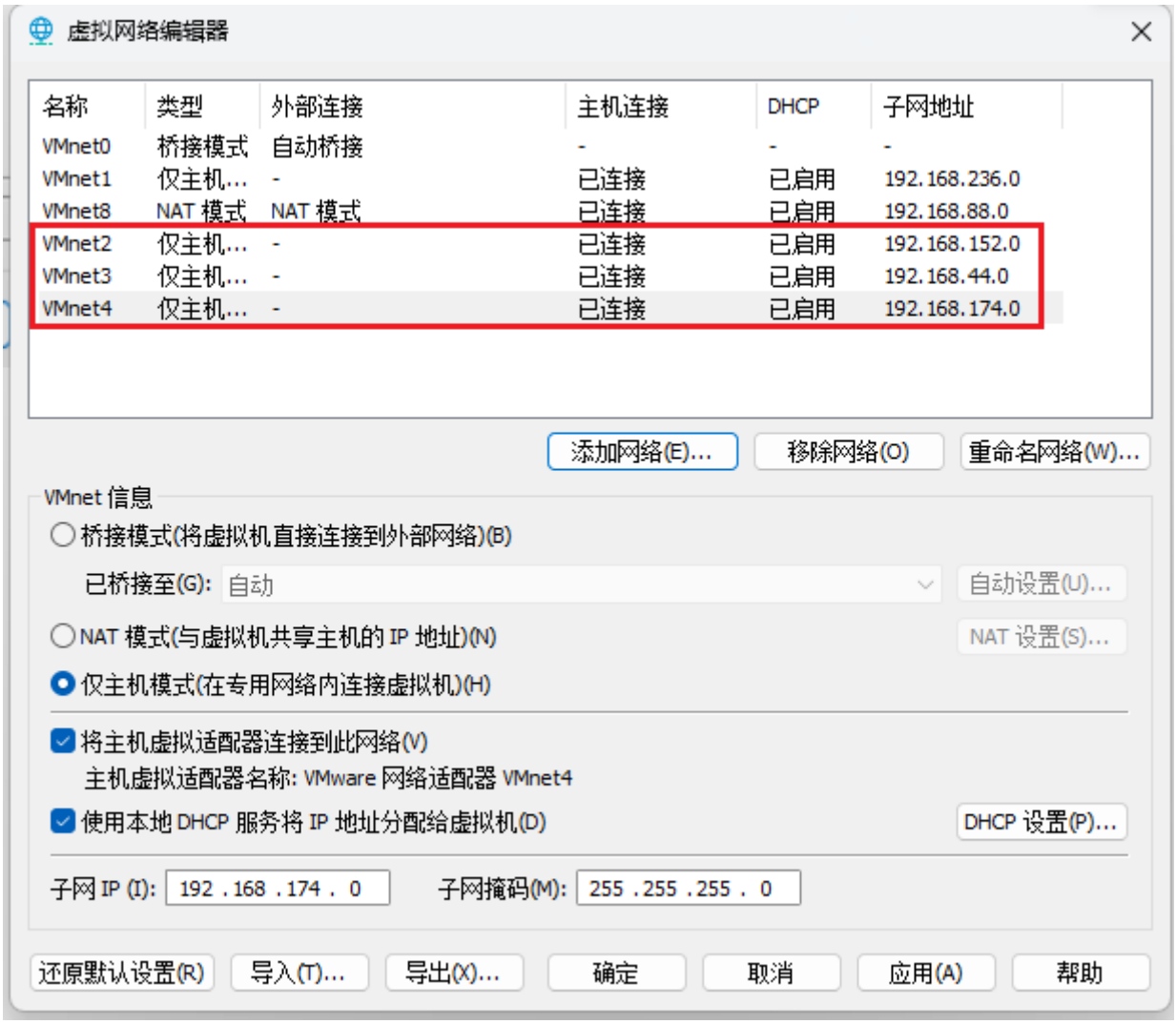
- 1- 配置VMware的网络编辑器， 创建出三个子网络：
  - 网络地址：192.168.1.0/24
    网络地址： 10.1.1.0/24
    网络地址：172.16.0.0/24







依次类推， 添加出三个虚拟网络



修改每个交换机的网络地址和对应子网掩码

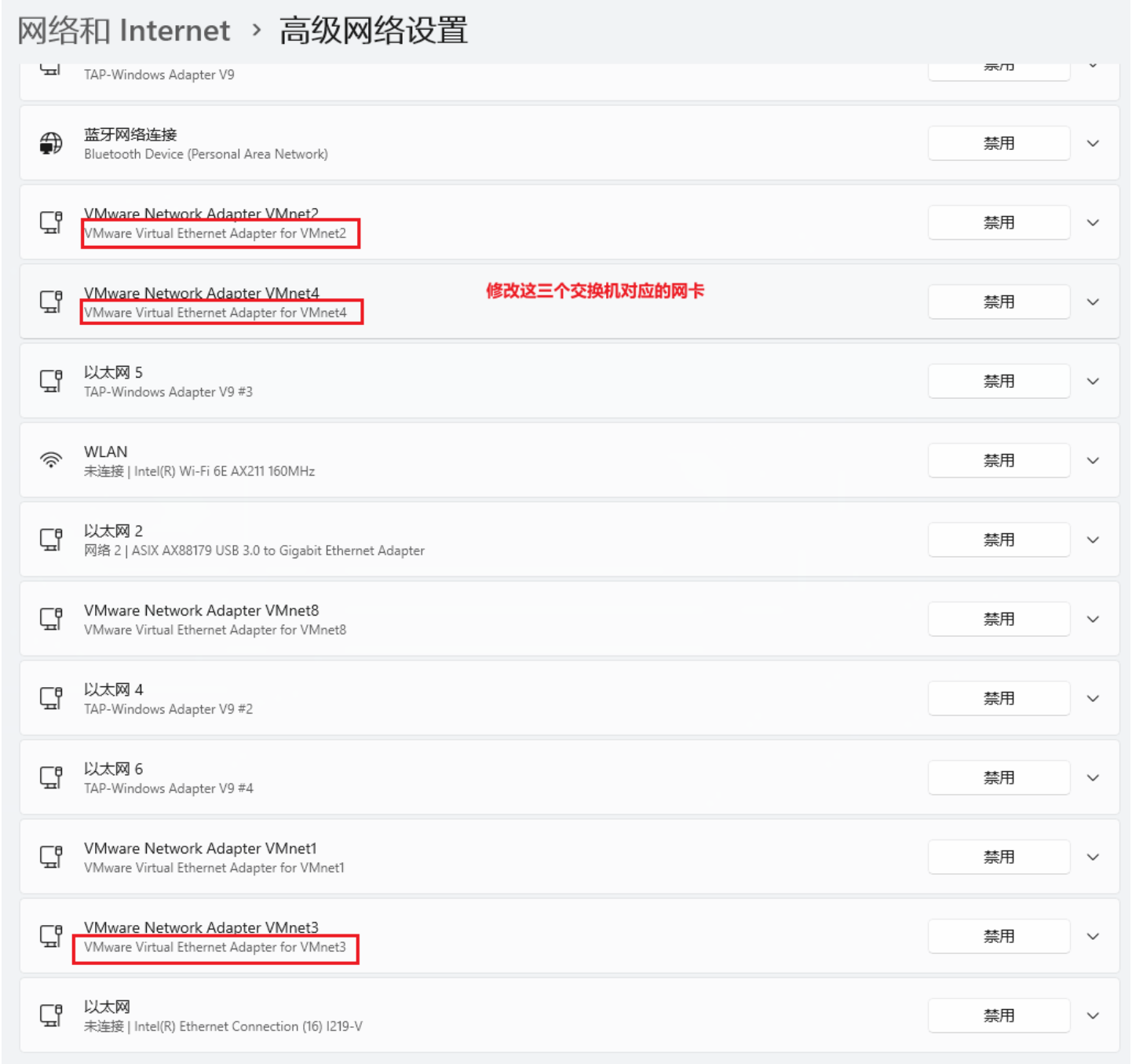

虚拟网络编辑器
名称 类型 外部连接 主机连接 DHCP 子网地址
VMnet0 桥接模式 自动桥接 - - -
VMnet1 仅主机... - 已连接 已启用 192.168.236.0
VMnet8 NAT 模式 NAT 模式 已连接 已启用 192.168.88.0
VMnet2 仅主机... - 已连接 已启用 192.168.1.0
VMnet3 仅主机... - 已连接 已启用 192.168.44.0
VMnet4 仅主机... - 已连接 已启用 192.168.174.0
1
添加网络(E)...
移除网络(O)
重命名网络(W)...
VMnet 信息
桥接模式(将虚拟机直接连接到外部网络)(B)
已桥接至(G): 自动
自动设置(U)...
NAT 模式(与虚拟机共享主机的 IP 地址)(N)
NAT 设置(S)...
仅主机模式(在专用网络内连接虚拟机)(H)
将主机虚拟适配器连接到此网络(V)
2
主机虚拟适配器名称: VMware 网络适配器 VMnet2
使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)
确保交换机可以进行IP分配任务
DHCP 设置(P)...
子网 IP (I): 192 . 168 . 1 . 0
3
子网掩码(M): 255 . 255 . 255 . 0
4
5
还原默认设置(R)
导入(T)...
导出(X)...
确定
取消
应用(A)
帮助


虚拟网络编辑器
名称 类型 外部连接 主机连接 DHCP 子网地址
VMnet0 桥接模式 自动桥接 - - -
VMnet1 仅主机... - 已连接 已启用 192.168.236.0
VMnet8 NAT 模式 NAT 模式 已连接 已启用 192.168.88.0
VMnet2 仅主机... - 已连接 已启用 192.168.1.0
VMnet3 仅主机... - 已连接 已启用 10.1.1.0
VMnet4 仅主机... - 已连接 已启用 192.168.174.0
1
添加网络(E)...
移除网络(O)
重命名网络(W)...
VMnet 信息
桥接模式(将虚拟机直接连接到外部网络)(B)
已桥接至(G): 自动
自动设置(U)...
NAT 模式(与虚拟机共享主机的 IP 地址)(N)
NAT 设置(S)...
仅主机模式(在专用网络内连接虚拟机)(H)
将主机虚拟适配器连接到此网络(V)
主机虚拟适配器名称: VMware 网络适配器 VMnet3
使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)
2
DHCP 设置(P)...
子网 IP (I): 10 . 1 . 1 . 0
3
子网掩码(M): 255 . 255 . 255 . 0
4
5
还原默认设置(R)
导入(T)...
导出(X)...
确定
取消
应用(A)
帮助

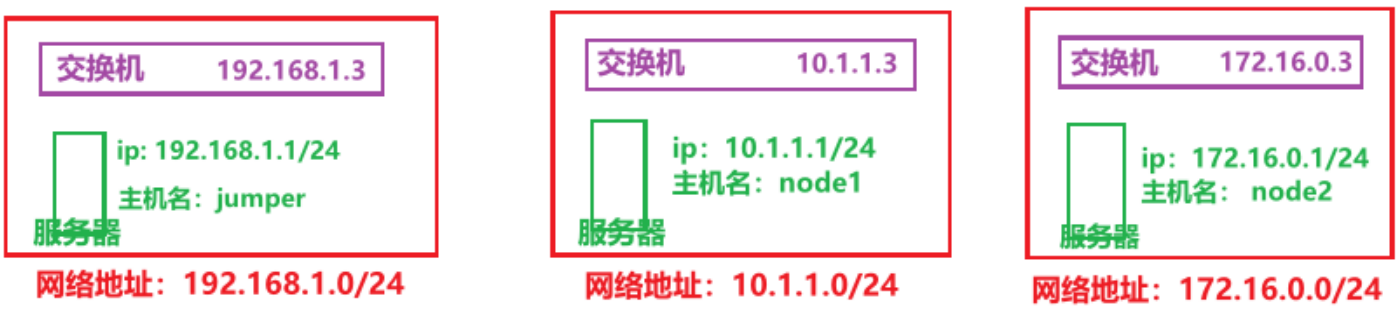


全部修改完成后， 点击确认即可

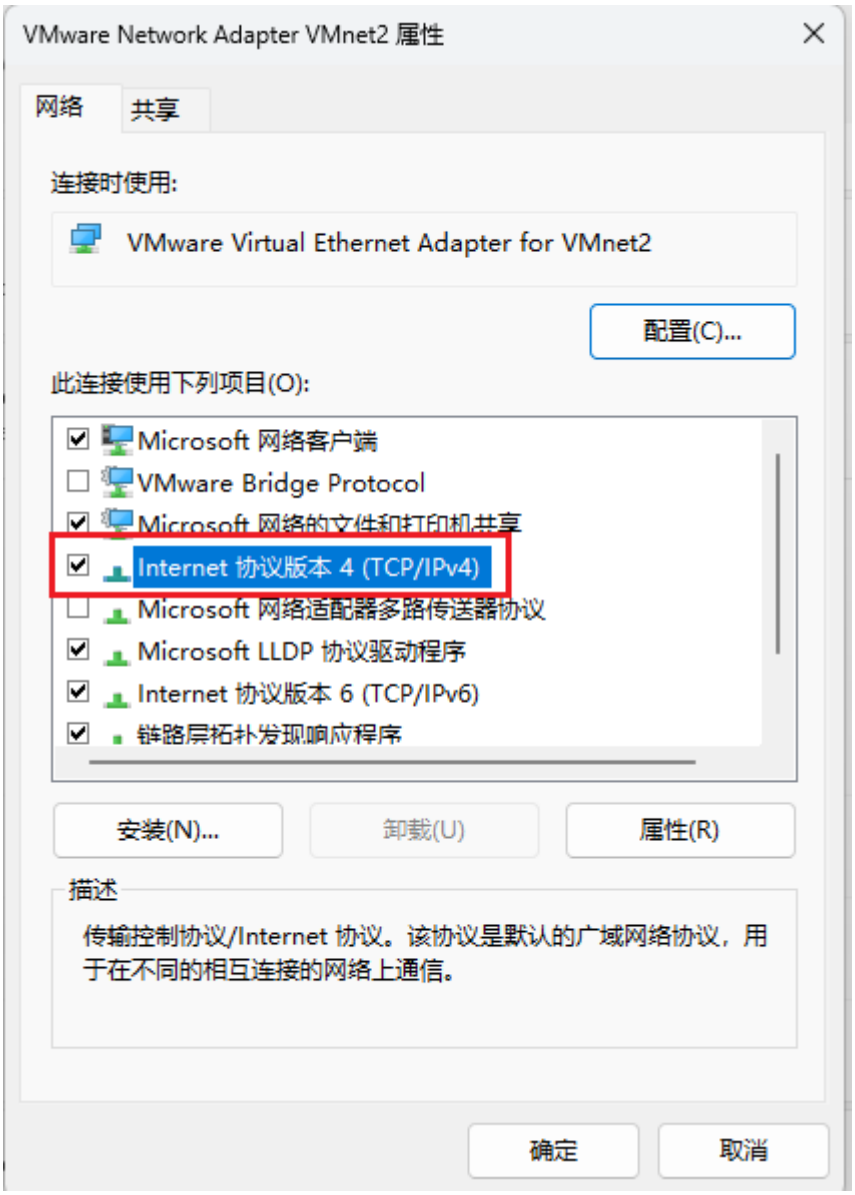
- 2- 修改交换机的IP地址

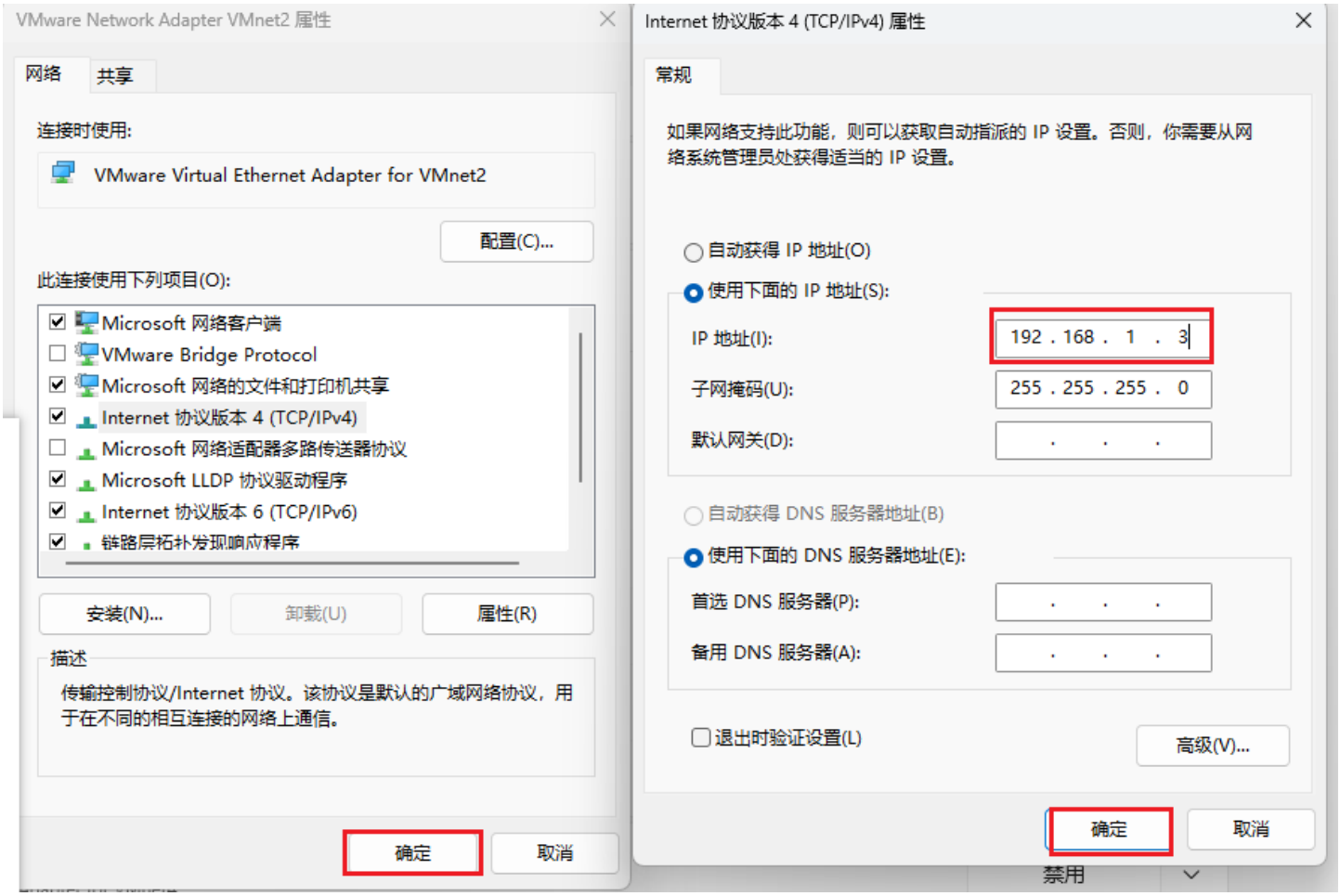


这三个交换机的IP地址为：



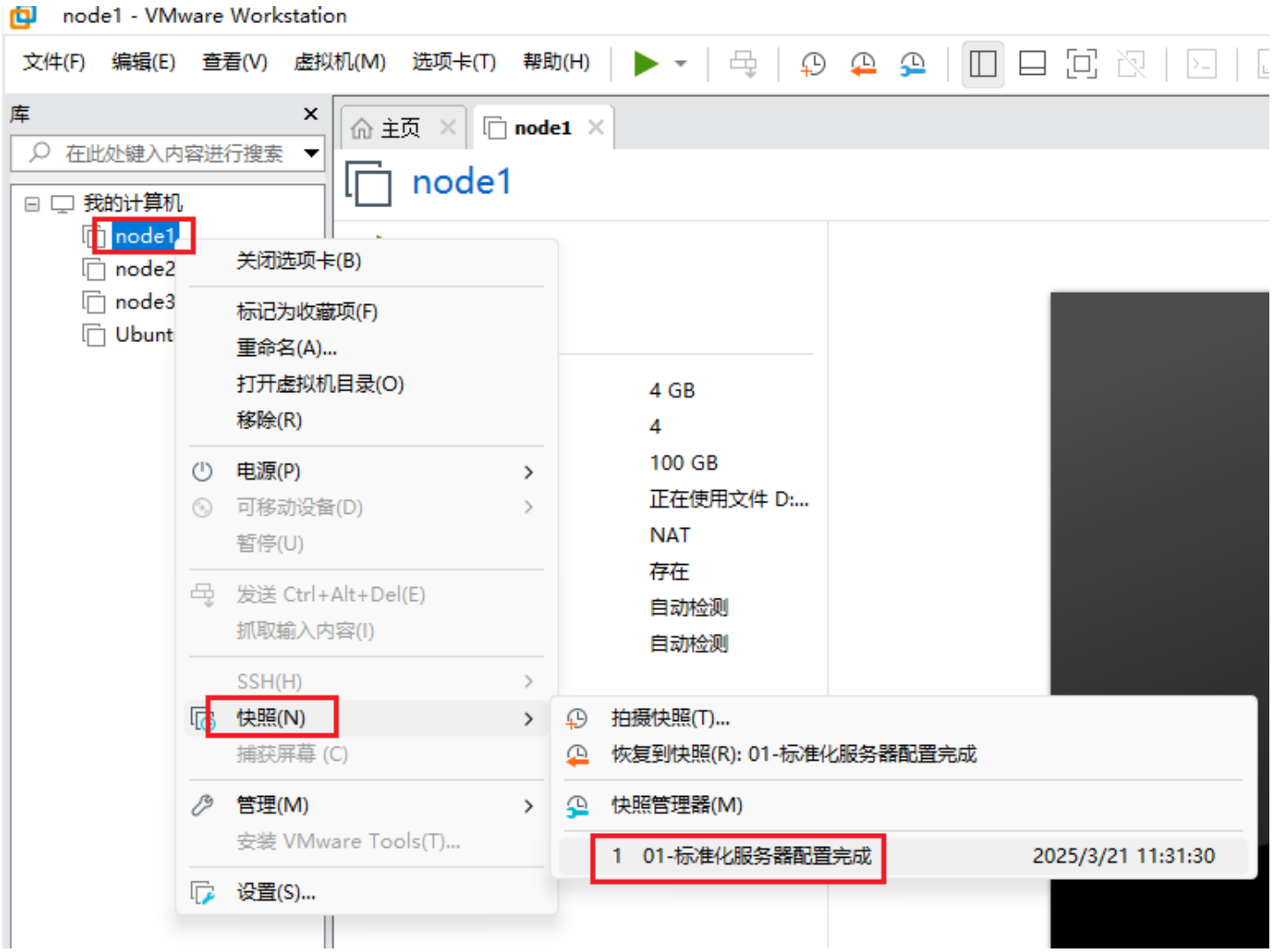

VMware Network Adapter VMnet2
VMware Virtual Ethernet Adapter fo VMnet2
以修改vmnet2为例
禁用
媒体状态: 已启用
发送的字节数: 342
接收的字节数: 726
链接速度: 100 (Mbps)
持续时间: 00:02:45
重命名此适配器
重命名
查看其他属性
更多适配器选项
编辑


VMware Network Adapter VMnet2 属性
网络
共享
连接时使用:
VMware Virtual Ethernet Adapter for VMnet2
配置(C)...
此连接使用下列项目(O):
Microsoft 网络客户端
VMware Bridge Protocol
Microsoft 网络的文件和打印机共享
Internet 协议版本 4 (TCP/IPv4)
Microsoft 网络适配器多路传送器协议
Microsoft LLDP 协议驱动程序
Internet 协议版本 6 (TCP/IPv6)
链路层拓扑发现响应程序
安装(N)...
卸载(U)
属性(R)
描述
传输控制协议/Internet 协议。该协议是默认的广域网络协议，用于在不同的相互连接的网络上通信。
确定
取消

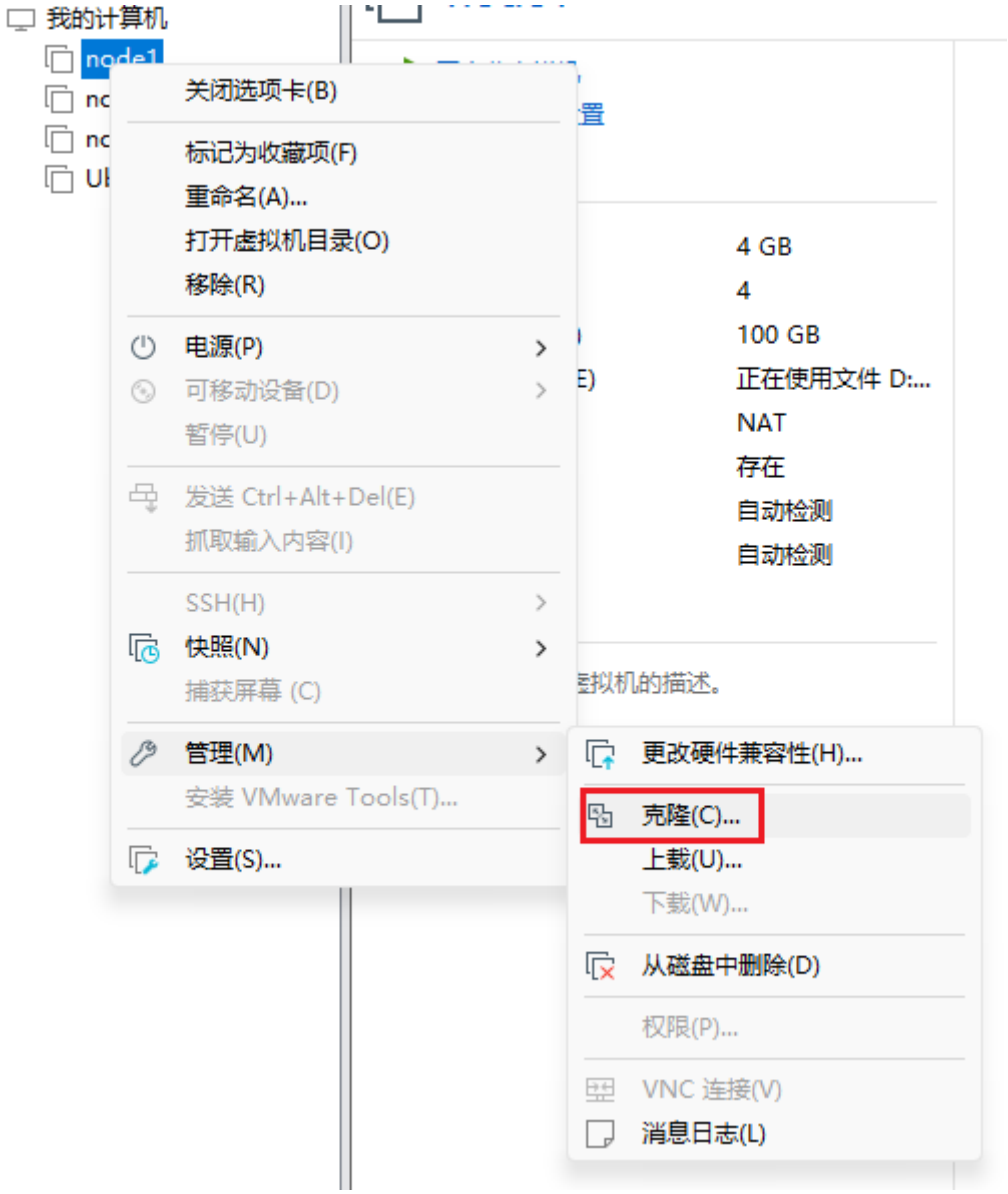


其他两张网卡是类似的， 分别修改为对应的交换机的IP地址即可

- 3- 通过克隆形成三个服务器： 全部都基于node1即可 **【前提请将node1恢复为标准的服务器快照】**


node1 - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
库
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubunt
主页
node1
node1
关闭选项卡(B)
标记为收藏项(F)
重命名(A)...
打开虚拟机目录(O)
移除(R)
电源(P)
可移动设备(D)
暂停(U)
发送 Ctrl+Alt+Del(E)
抓取输入内容(I)
SSH(H)
快照(N)
捕获屏幕 (C)
管理(M)
安装 VMware Tools(T)...
设置(S)...
4 GB
4
100 GB
正在使用文件 D:...
NAT
存在
自动检测
自动检测
拍摄快照(T)...
恢复到快照(R): 01-标准化服务器配置完成
快照管理器(M)
1 01-标准化服务器配置完成 2025/3/21 11:31:30

执行克隆操作：



通过克隆形成三个服务器， 分别将其名称命名为： jumper node1_copy node2_copy



- 4- 将每一台主机和对应的子网络交换机进行连接

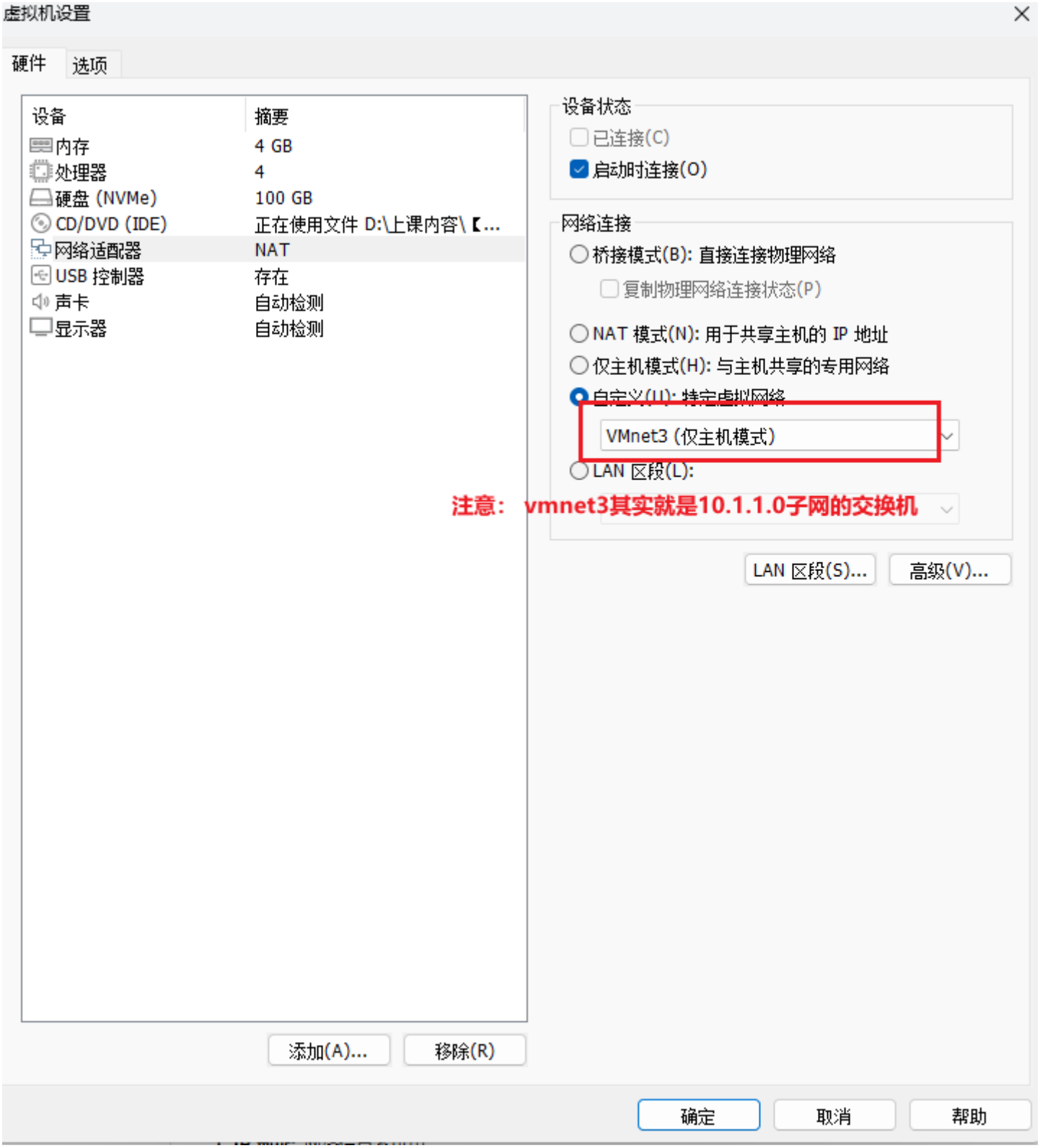
以jumper为例


库
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubuntu_Server_24.04
jumper
node1_copy
node2_copy
主页
node1
jumper
node1_copy
jumper
开启此虚拟机
编辑虚拟机设置
设备
内存 4 GB
处理器 4
硬盘 (NVMe) 100 GB
CD/DVD (IDE) 正在使用文件 D:...
网络适配器 NAT
USB 控制器 存在
声卡 自动检测
显示器 自动检测
描述
在此处键入对该虚拟机的描述。


虚拟机设置
硬件
选项
设备
摘要
内存
4 GB
处理器
4
硬盘 (NVMe)
100 GB
CD/DVD (IDE)
正在使用文件 D:\上课内容\...
网络适配器
NAT
USB 控制器
存在
声卡
自动检测
显示器
自动检测
添加(A)...
移除(R)
设备状态
已连接(C)
启动时连接(O)
网络连接
桥接模式(B): 直接连接物理网络
复制物理网络连接状态(P)
NAT 模式(N): 用于共享主机的 IP 地址
仅主机模式(H): 与主机共享的专用网络
自定义(U): 特定虚拟网络
VMnet2 (仅主机模式)
LAN 区段(L):
注意， 我这里vmnet2其实就是192.168.1.0 网络的对应交换机
LAN 区段(S)...
高级(V)...
确定
取消
帮助

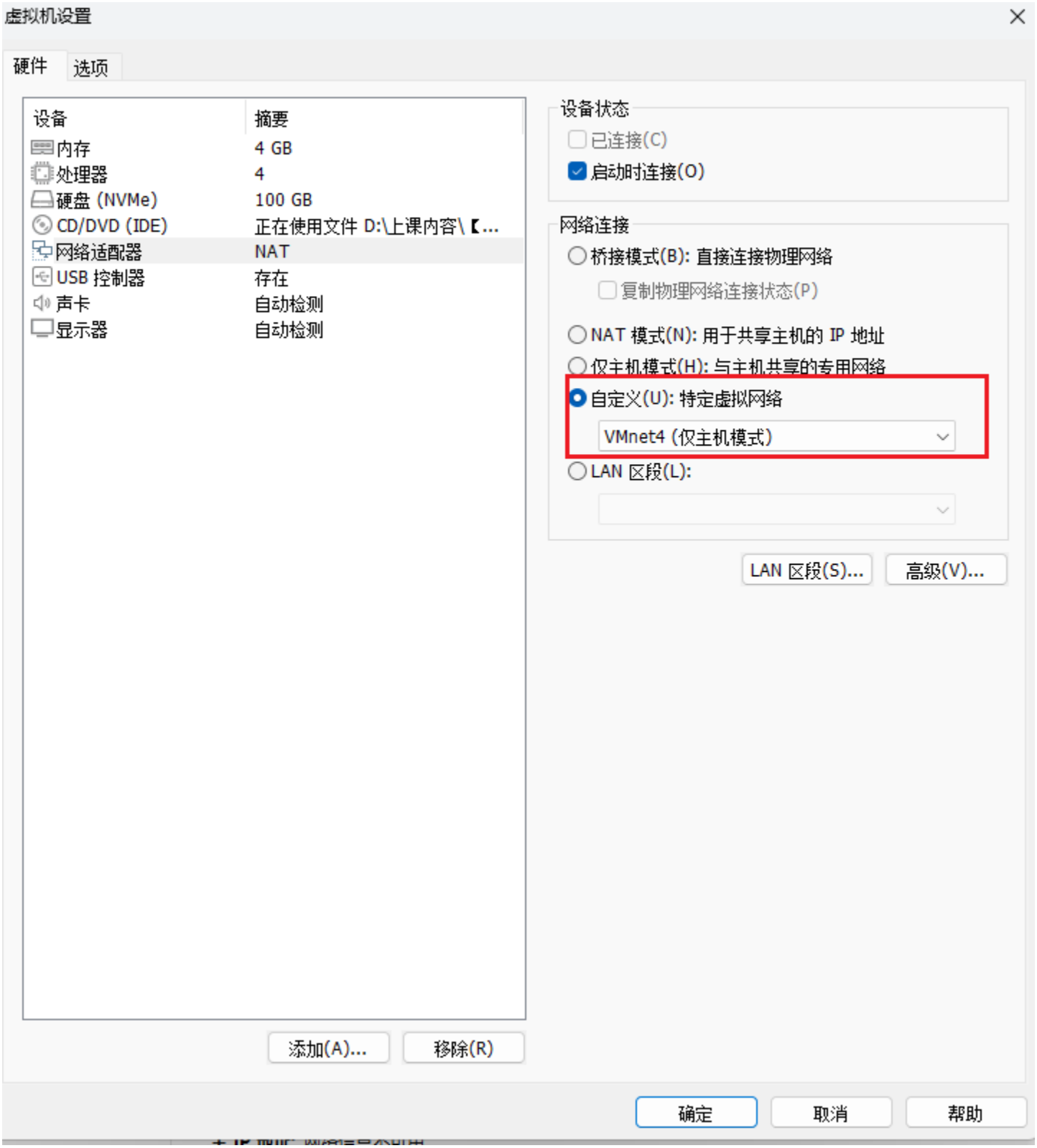
node1：


文件(F)
编辑(E)
查看(V)
虚拟机(M)
选项卡(T)
帮助(H)
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubuntu_Server_24.04
jumper
node1_copy
node2_copy
主页
node1
jumper
node1_copy
node2_copy
node1_copy
开启此虚拟机
编辑虚拟机设置
设备
内存 4 GB
处理器 4
硬盘 (NVMe) 100 GB
CD/DVD (IDE) 正在使用文件 D:...
网络适配器 NAT
USB 控制器 存在
声卡 自动检测
显示器 自动检测
描述
在此处键入对该虚拟机的描述。


虚拟机设置
硬件
选项
设备
摘要
内存
4 GB
处理器
4
硬盘 (NVMe)
100 GB
CD/DVD (IDE)
正在使用文件 D:\上课内容\...
网络适配器
NAT
USB 控制器
存在
声卡
自动检测
显示器
自动检测
添加(A)...
移除(R)
设备状态
已连接(C)
启动时连接(O)
网络连接
桥接模式(B): 直接连接物理网络
复制物理网络连接状态(P)
NAT 模式(N): 用于共享主机的 IP 地址
仅主机模式(H): 与主机共享的专用网络
自定义(U): 特定虚拟网络
VMnet3 (仅主机模式)
LAN 区段(L):
注意： vmnet3其实就是10.1.1.0子网的交换机
LAN 区段(S)...
高级(V)...
确定
取消
帮助

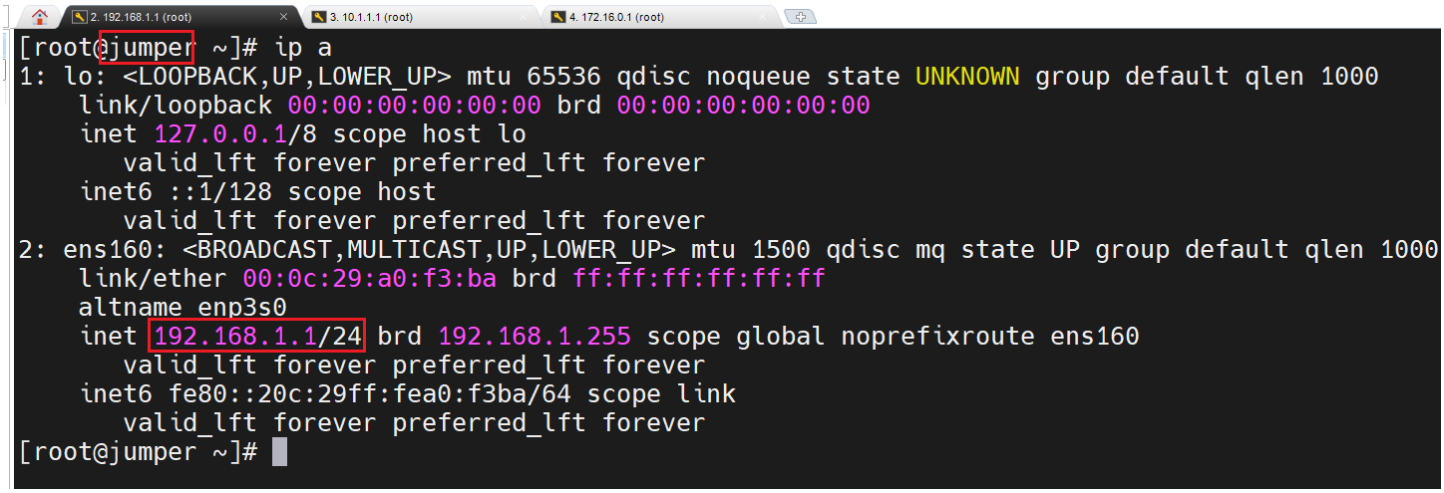
node2

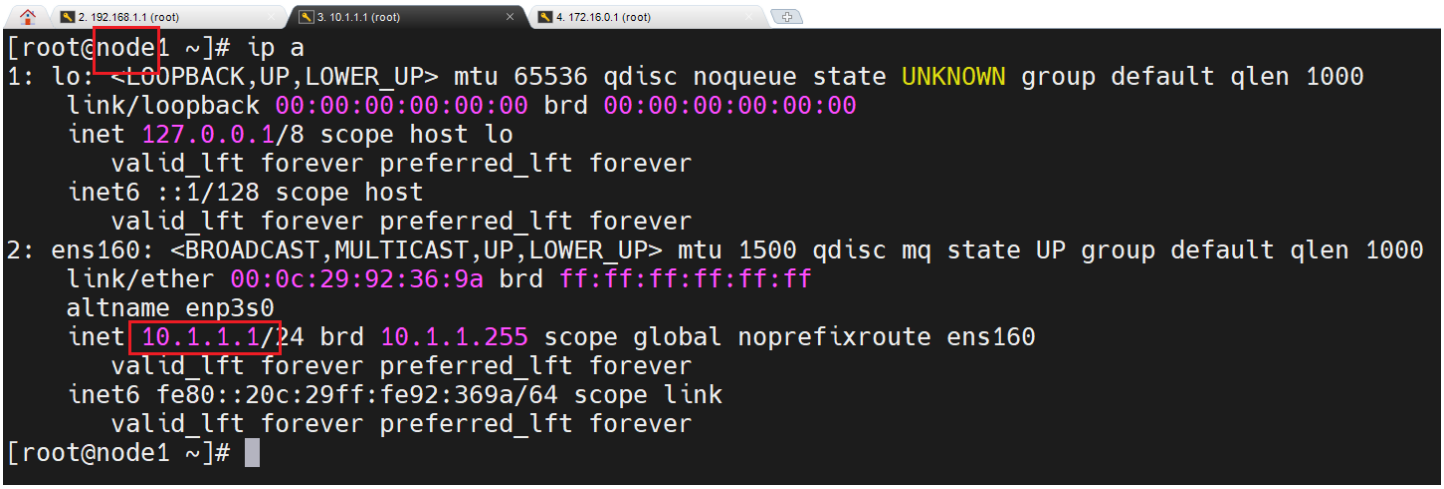

文件(F)
编辑(E)
查看(V)
虚拟机(M)
选项卡(T)
帮助(H)
库
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubuntu_Server_24.04
jumper
node1_copy
node2_copy
主页
node1
jumper
node1_copy
node2_copy
node2_copy
开启此虚拟机
编辑虚拟机设置
设备
内存 4 GB
处理器 4
硬盘 (NVMe) 100 GB
CD/DVD (IDE) 正在使用文件 D:...
网络适配器 NAT
USB 控制器 存在
声卡 自动检测
显示器 自动检测
描述
在此处键入对该虚拟机的描述。

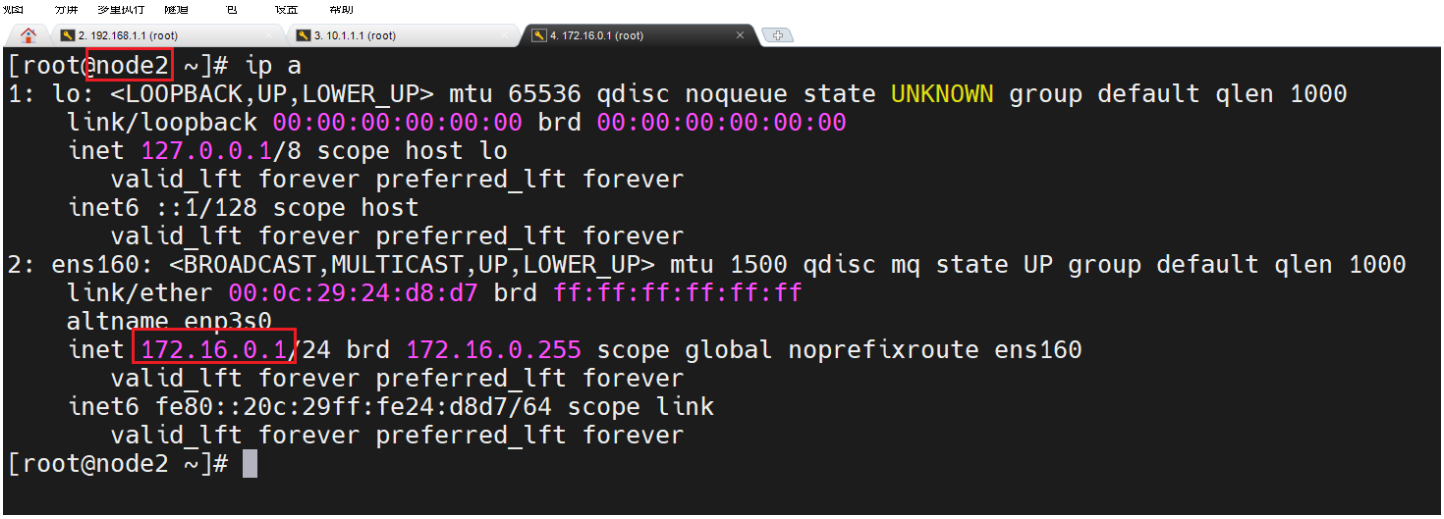


注意：一定要更改为属于自己的对应网卡

- 第五步：开启每一台服务器， 修改其静态的IP地址
- 第六步：使用MX工具连接每一台服务器
- 第七步： 修改每一台服务器的主机名

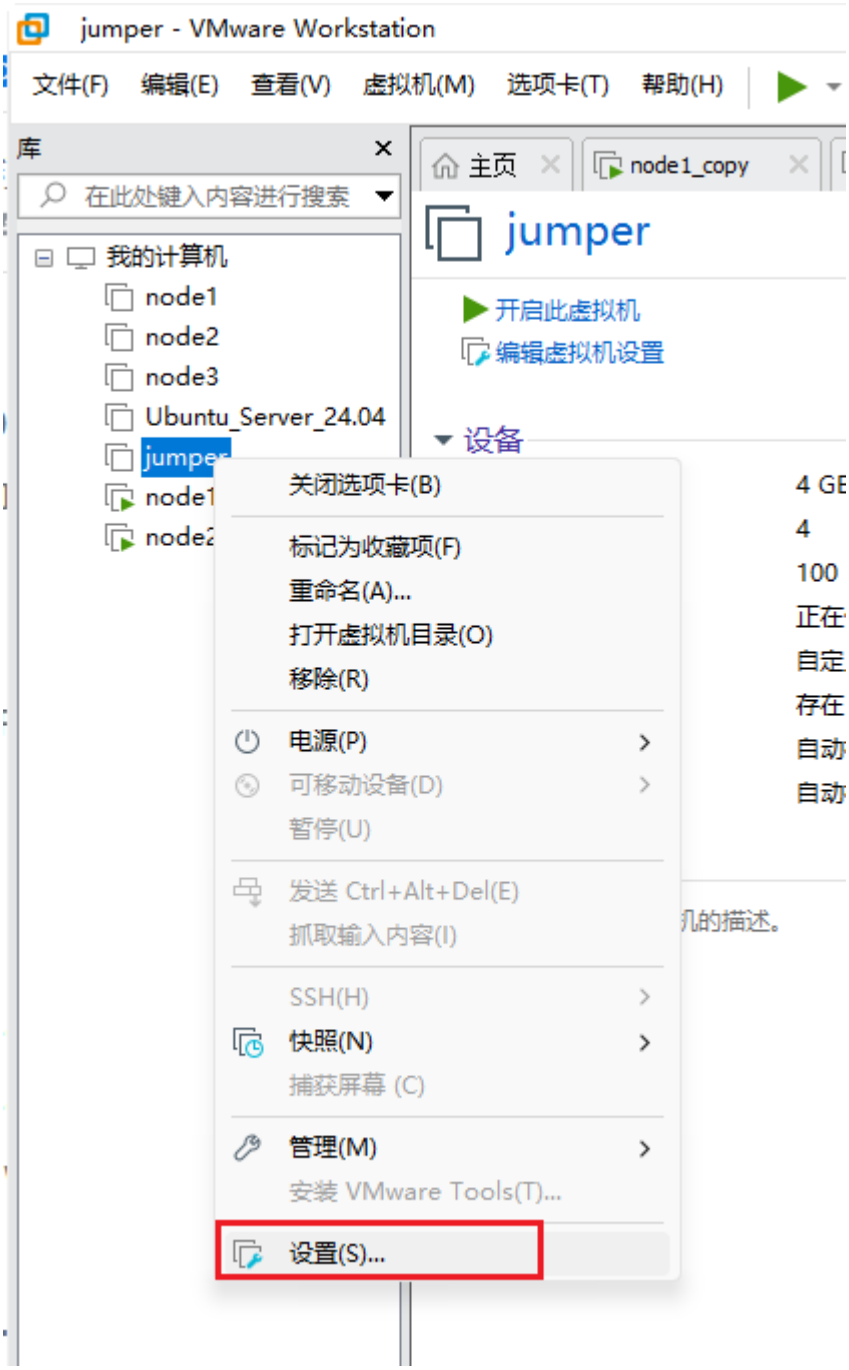


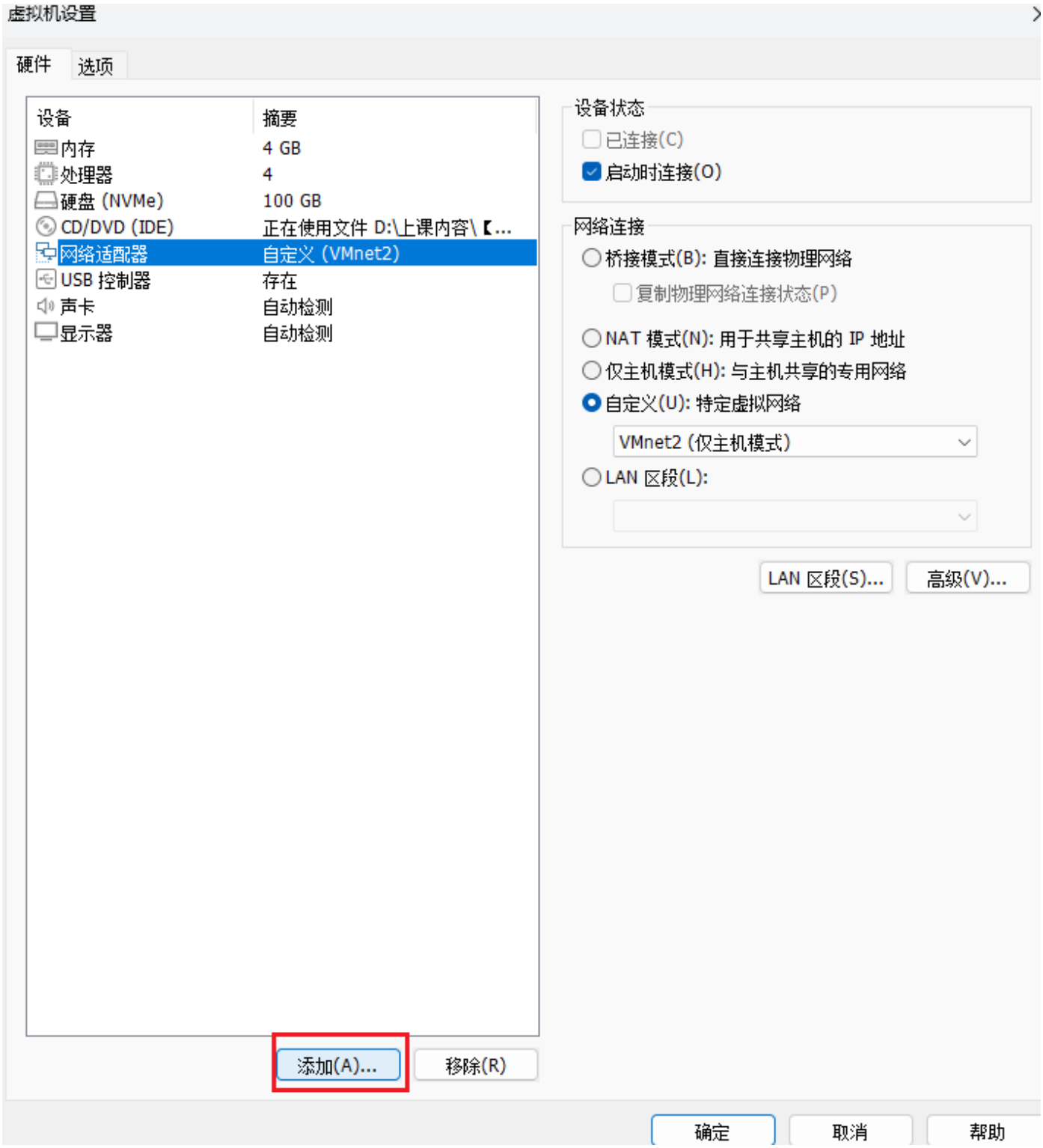




网络打通实操：

- 1- 在jumper的主机上额外安装二张网卡， 通过二张网卡分别和node1、node2所对应网络建立关联
  - 1.1 将jumper先关机
  - 1.2 添加网卡

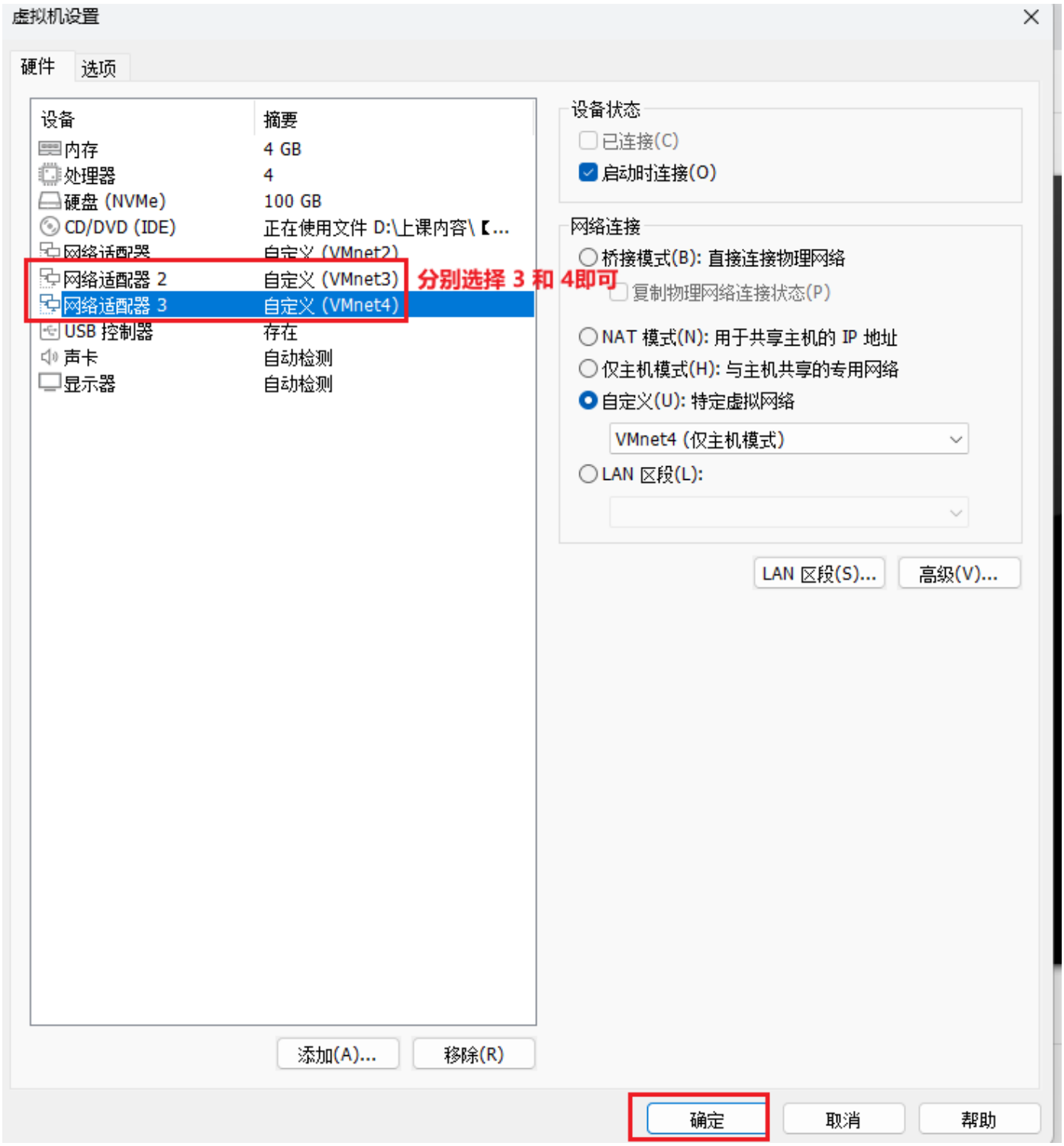

jumper - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
库
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubuntu_Server_24.04
jumper
node1
node2
主页
node1_copy
jumper
开启此虚拟机
编辑虚拟机设置
设备
4 GE
4
100
正在
自定
存在
自动
自动
关闭选项卡(B)
标记为收藏项(F)
重命名(A)...
打开虚拟机目录(O)
移除(R)
电源(P)
可移动设备(D)
暂停(U)
发送 Ctrl+Alt+Del(E)
抓取输入内容(I)
SSH(H)
快照(N)
捕获屏幕 (C)
管理(M)
安装 VMware Tools(T)...
设置(S)...


虚拟机设置
硬件
选项
设备
摘要
内存
4 GB
处理器
4
硬盘 (NVMe)
100 GB
CD/DVD (IDE)
正在使用文件 D:\上课内容\【...
网络适配器
自定义 (VMnet2)
USB 控制器
存在
声卡
自动检测
显示器
自动检测
添加(A)...
移除(R)
设备状态
已连接(C)
启动时连接(O)
网络连接
桥接模式(B): 直接连接物理网络
复制物理网络连接状态(P)
NAT 模式(N): 用于共享主机的 IP 地址
仅主机模式(H): 与主机共享的专用网络
自定义(U): 特定虚拟网络
VMnet2 (仅主机模式)
LAN 区段(L):
LAN 区段(S)...
高级(V)...
确定
取消
帮助



- 1.3 修改网络适配器要连接的交换机

  注意： 一张连接node1所对应的网卡 一张连接node2所对应的网卡



- 1.4 开机， 使用MX工具连接后， 查看是否有其他二个网络的IP地址

```
[root@jumper ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
主网卡 valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:a0:f3:ba brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.1.1/24 brd 192.168.1.255 scope global noprefixroute ens160
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fea0:f3ba/64 scope link
辅网卡-1valid_lft forever preferred_lft forever
3: ens224: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:a0:f3:c4 brd ff:ff:ff:ff:ff:ff
    altname enp19s0
    inet 10.1.1.128/24 brd 10.1.1.255 scope global dynamic noprefixroute ens224
       valid_lft 1789sec preferred_lft 1789sec
    inet6 fe80::b76d:19dd:6e:274b/64 scope link noprefixroute
辅网卡-2valid_lft forever preferred_lft forever
4: ens256: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:a0:f3:ce brd ff:ff:ff:ff:ff:ff
    altname enp27s0
    inet 172.16.0.128/24 brd 172.16.0.255 scope global dynamic noprefixroute ens256
       valid_lft 1789sec preferred_lft 1789sec
    inet6 fe80::ce62:4718:c4b9:a916/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

此时， 在node1和node2 分别测试 访问对应网卡的IP地址即可，正常应该是可以ping通的，因为你们是属于同一个网络下的

在node1：

```
ping 10.1.1.128
```

在node2：

```
ping 172.16.0.128
```

正常都是可以ping通， 因为通过这两张网卡， 实现了让主机分别成为了对应子网络的一台主机设备

大家此时可以看下node1和node2的路由表， 在路由表中记录了，访问目标路径的路由信息

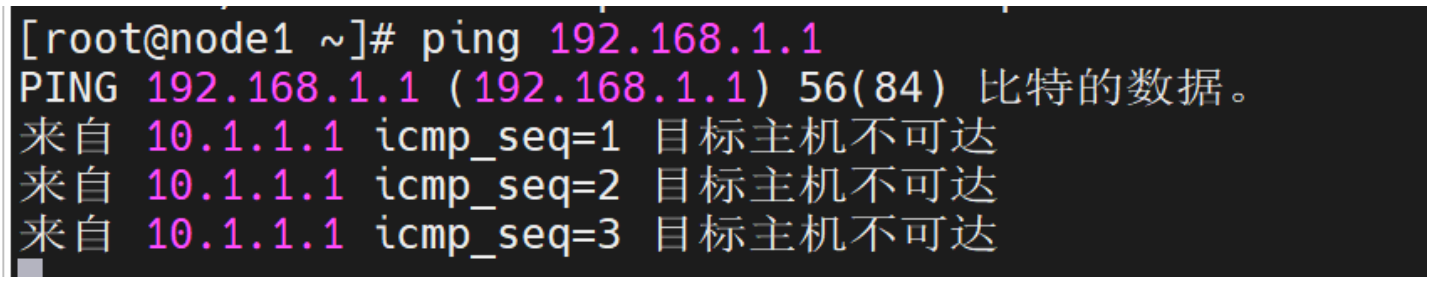
```
[root@node1 ~]# ping 10.1.1.128
PING 10.1.1.128 (10.1.1.128) 56(84) 比特的数据。
64 比特，来自 10.1.1.128: icmp_seq=1 ttl=64 时间=0.548 毫秒
64 比特，来自 10.1.1.128: icmp_seq=2 ttl=64 时间=0.456 毫秒
```

```
[root@node2 ~]# ping 172.16.0.128
PING 172.16.0.128 (172.16.0.128) 56(84) 比特的数据。
64 比特，来自 172.16.0.128: icmp_seq=1 ttl=64 时间=0.502 毫秒
64 比特，来自 172.16.0.128: icmp_seq=2 ttl=64 时间=0.344 毫秒
```

查看路由表：以其中一个为例



请思考： 如果现在让node1或者node2连接jumper主网卡的IP是否可以ping通呢?




```
原有：
    你ping的IP不是通网络的IP， 此时， 在路由表中， 就会走默认网关， 但是默认网关对应IP地址压根不存在

    所以通过这个网关你的流程出不去呀

所以如何解决呢？ 需要再node1和node2配置路由， 通过路由实现到jumper 主网卡的路径
    以node1连接jumper的主网卡为例
    方式一：
        采用主机路由：在node1中配置
            ip route add 192.168.1.1 via 10.1.1.128

            在node1访问192.168.1.1 ip时候， 会通过 10.1.1.128 网关实现数据联通

        采用网络路由： 在node1中配置
            # 先将主机路由删除， 以避免测试不准确
            ip route del 192.168.1.1
            # 添加
            ip route add 192.168.1.0/24 via 10.1.1.128

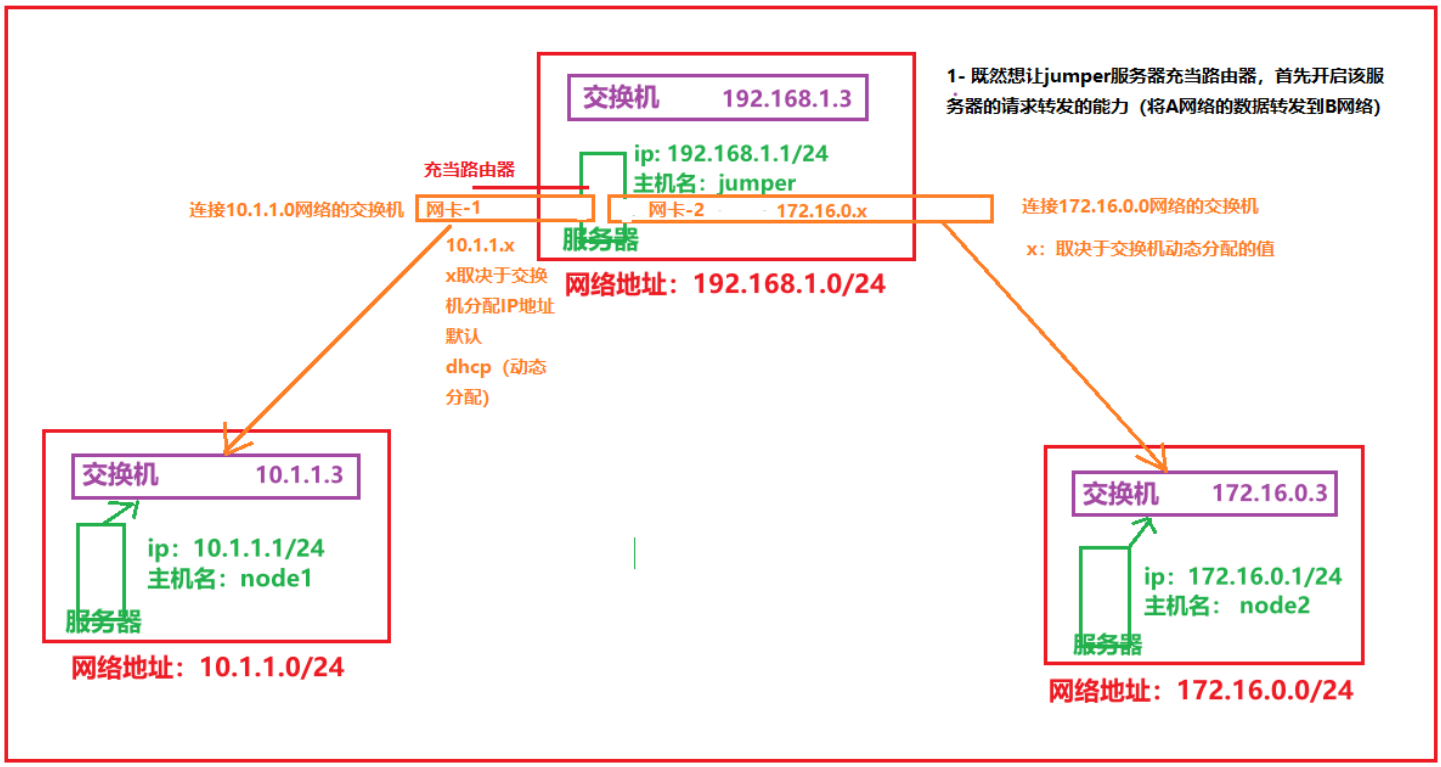
            在node1访问192.168.1.1 ip时候，会被网络路由识别到，然后会通过该路由信息中标记的 10.1.1.128 网关实现数据联通

        采用默认网关：
             # 先将主机网络路由删除， 以避免测试不准确
            ip route del 192.168.1.0/24
            # 删除原来的默认网关
```

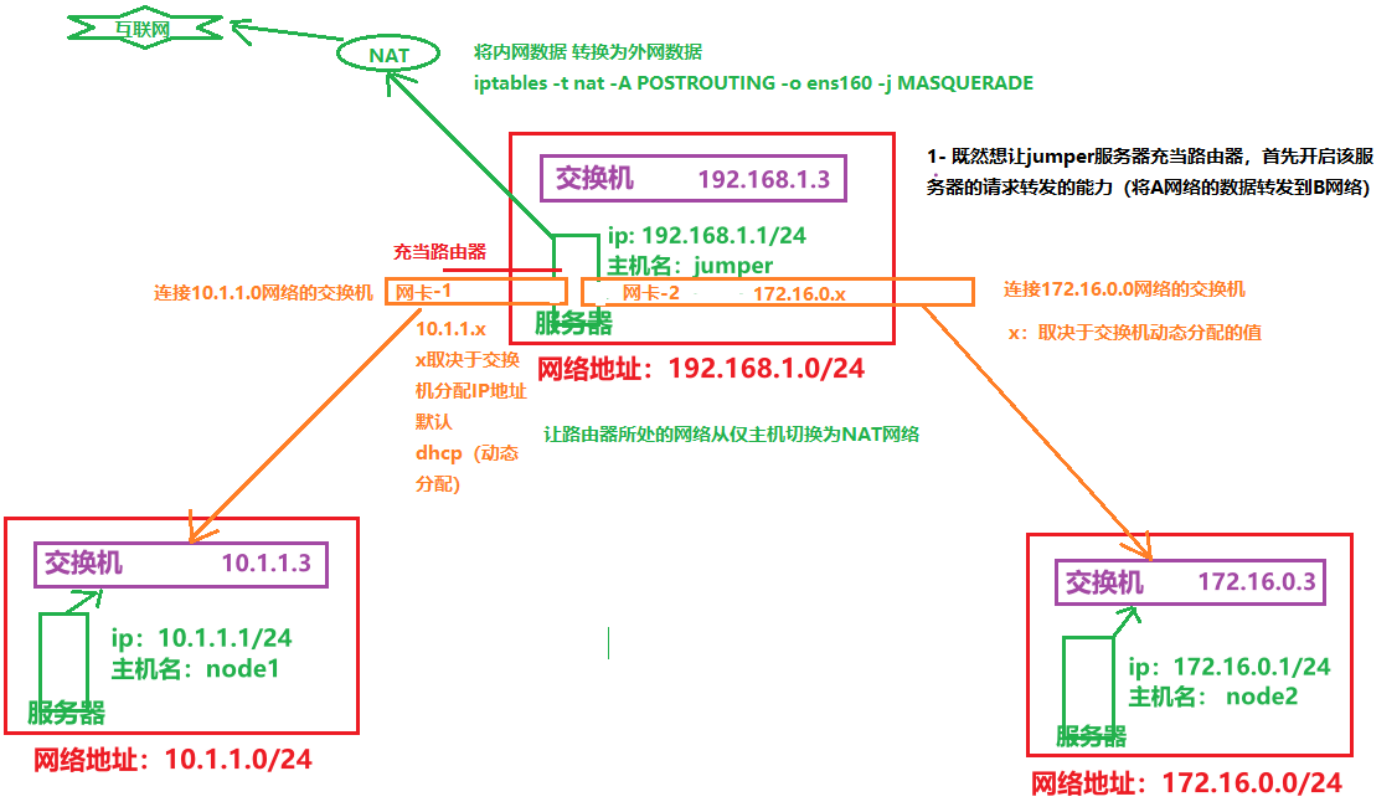
```
            ip route del default via 10.1.1.2  # 这个ip地址可能每个人不一样，或者压根不存在默认网关
            # 添加默认网关
            ip route add default via 10.1.1.128

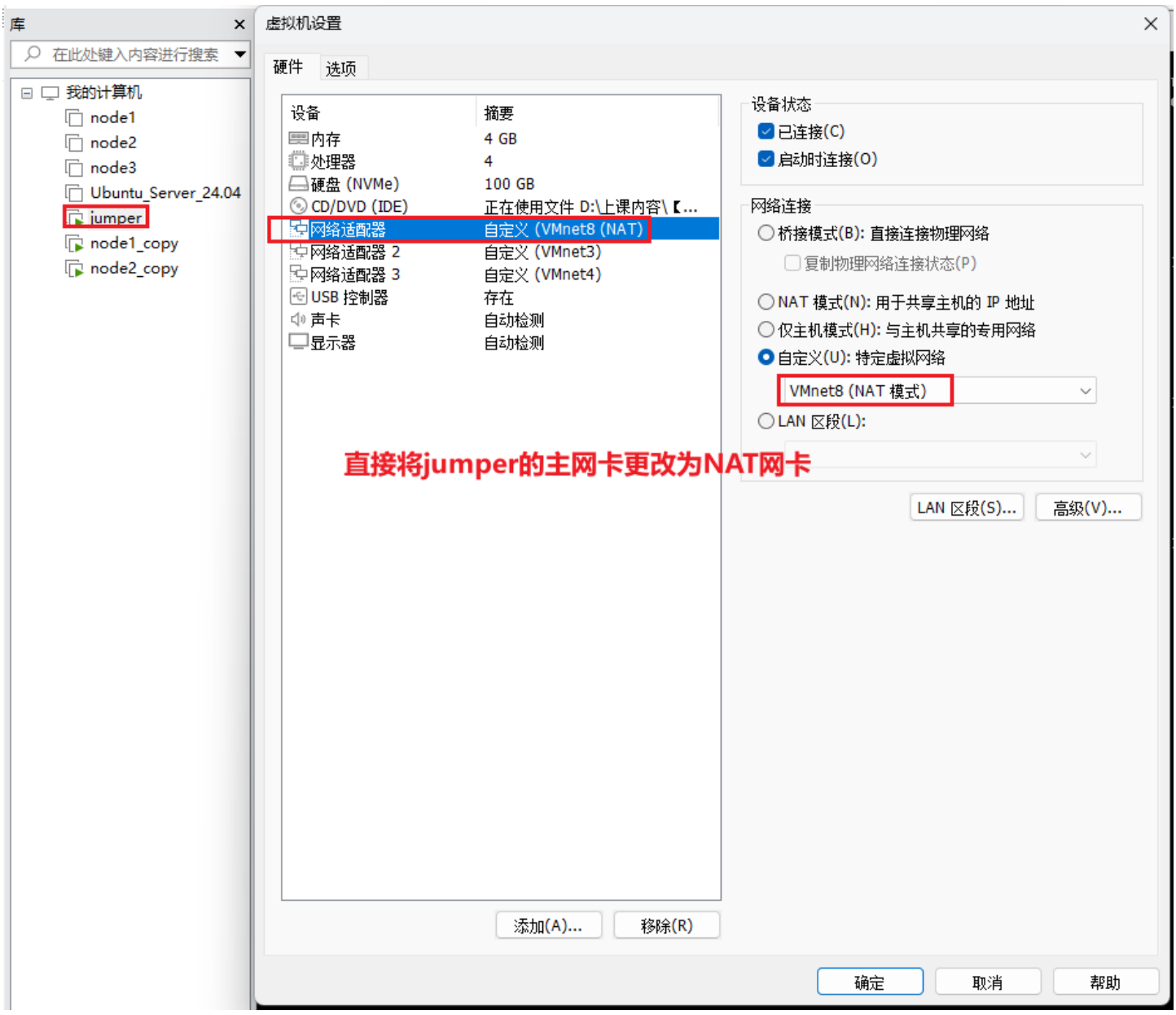

当能够理解此处的路径的配置的时候， 其实呀， node1到node2的网络的路由你就会配置了。。。。

注意： 记得在jumper中开启包转发的能力
```



如何基于NAT主机,使仅主机模式连接外部网络:


互联网
NAT
将内网数据 转换为外网数据
iptables -t nat -A POSTROUTING -o ens160 -j MASQUERADE
交换机 192.168.1.3
1- 既然想让jumper服务器充当路由器，首先开启该服务器的请求转发的能力（将A网络的数据转发到B网络）
ip: 192.168.1.1/24
主机名：jumper
充当路由器
连接10.1.1.0网络的交换机
网卡-1
网卡-2
172.16.0.x
连接172.16.0.0网络的交换机
x：取决于交换机动态分配的值
服务器
10.1.1.x
x取决于交换机分配IP地址
默认
dhcp（动态分配）
网络地址：192.168.1.0/24
让路由器所处的网络从仅主机切换为NAT网络
交换机 10.1.1.3
ip：10.1.1.1/24
主机名：node1
服务器
网络地址：10.1.1.0/24
交换机 172.16.0.3
ip：172.16.0.1/24
主机名： node2
服务器
网络地址：172.16.0.0/24


库
在此处键入内容进行搜索
我的计算机
node1
node2
node3
Ubuntu_Server_24.04
jumper
node1_copy
node2_copy
虚拟机设置
硬件
选项
设备
摘要
内存 4 GB
处理器 4
硬盘 (NVMe) 100 GB
CD/DVD (IDE) 正在使用文件 D:\上课内容\...
网络适配器 自定义 (VMnet8 (NAT))
网络适配器 2 自定义 (VMnet3)
网络适配器 3 自定义 (VMnet4)
USB 控制器 存在
声卡 自动检测
显示器 自动检测
添加(A)...
移除(R)
设备状态
已连接(C)
启动时连接(O)
网络连接
桥接模式(B): 直接连接物理网络
复制物理网络连接状态(P)
NAT 模式(N): 用于共享主机的 IP 地址
仅主机模式(H): 与主机共享的专用网络
自定义(U): 特定虚拟网络
VMnet8 (NAT 模式)
LAN 区段(L):
LAN 区段(S)...
高级(V)...
直接将jumper的主网卡更改为NAT网卡
确定
取消
帮助

- 第四步- 修改jumper中网卡设备信息， 确保能够连接这个能够上网的网关

  vi /etc/NetworkManager/system-connections/ens160.nmconnection

```
[connection]
id=ens160
uuid=32878fdb-b450-3d07-bece-bc533ac66c3e
type=ethernet
autoconnect-priority=-999
interface-name=ens160
timestamp=1741804943

[ethernet]

[ipv4]
method=manual
addresses=192.168.88.125/24
gateway=192.168.88.2
dns=8.8.8.8;114.114.114.114;

[ipv6]
addr-gen-mode=eui64
method=ignore

[proxy]
~
```

修改为88网段的ip和网关

  重启网卡配置：

  systemctl restart NetworkManager

最后在jumper中，执行：

```
iptables -t nat -A POSTROUTING -o ens160 -j MASQUERADE
```

测试：

```
首先： 在node1和node2中， 将默认网关配置为jumper对应的辅网卡
node1： ip route add default via 10.1.1.128

node2: ip route add default via 172.16.0.128

最后通过： ping www.baidu.com
```

```
关于内网转外网设置， 需要修改 NAT 规则（使用 iptables）
iptables -t nat -A POSTROUTING -o ens160 -j MASQUERADE
-t nat：操作 NAT 表。
-A POSTROUTING：修改出站流量。
-o 160：匹配外网接口。
MASQUERADE：动态转换源 IP。

内网私有地址（如 192.168.x.x）：无法直接在公网中使用，需要 NAT 转换为路由器的对外 IP。
NAT 的作用：将内网私有地址和公网地址之间建立映射，使得内网主机可以与外网通信。
没有 NAT 规则：
内网主机的数据包会到达 NAT 路由器（node2），但无法被正确转换和转发到外网。
外网返回的数据包无法找到内网主机。


确认规则已添加并找到对应的规则编号
iptables -t nat -L POSTROUTING --line-numbers -n -v
删除规则：
iptables -t nat -D POSTROUTING <规则编号>
查看流量统计信息
iptables -t nat -L -v
```

新增网卡，在虚拟机中没有配置文件， 可以通过下面的命令生成配置文件

nmcli connection add type ethernet con-name <连接名称> ifname <网卡名称>

```
例如： 增加 ens256网卡配置
nmcli connection add type ethernet con-name ens256 ifname ens256
```